



**Hochschule für Technik
und Wirtschaft Berlin**

University of Applied Sciences

Design of function-calling agents in Unity using LLMs and RAG

Masterarbeit

Name des Studiengangs

Internationale Medieninformatik

Fachbereich 4 – Informatik, Kommunikation und Wirtschaft –

vorgelegt von

Ruslan Novikov

Tag der Abgabe: 21.02.2025

Erstgutachten: Prof. Dr. Tobias Lenz

Zweitgutachten: Prof. Dr.-Ing. David Strippgen

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Research Scope and Objectives	1
1.3	Technical Innovation	2
1.4	Research Methodology	2
1.5	Scope Limitations	3
1.6	Thesis Structure	3
1.7	Expected Contributions	3
2	Background and Literature Review	5
2.1	Large Language Models	5
2.1.1	Understanding Language Models	5
2.1.2	Core Capabilities and Limitations	6
2.2	LLMs as Agents	7
2.2.1	Agent Fundamentals	7
2.2.2	Architectural Framework	8
2.3	Enhancing Memory through Knowledge Integration	11
2.3.1	Retrieval-Augmented Generation	11
2.3.2	Knowledge Graphs as Structured Memory	13
2.3.3	Basic Concepts and Definition	14
2.3.4	Graph Structures and Models	14
2.3.5	Data Retrieval	15
2.4	Planning and Action Frameworks	18
2.4.1	Prompt Engineering	19
2.4.2	Reasoning frameworks	20
3	Methodology	23
3.1	Research approach	23
3.2	Implementation process	23
4	Analysis	25
4.1	Analysis of Contemporary Solutions	25
4.2	Implementation Analysis	27
4.2.1	Agent Framework Implementations	27
4.2.2	Evaluation Analysis	32
4.2.3	Key Findings and Limitations	36

5	Implementation	39
5.1	System Architecture	39
5.1.1	Core Components	39
5.1.2	Technical Stack	40
5.2	Knowledge Graph	41
5.2.1	Design Considerations	41
5.2.2	Schema Design	41
5.2.3	Relationships	43
5.2.4	Validation	44
5.2.5	Data Collection	45
5.2.6	Semantic Enhancement	47
5.3	Method Invocation System	47
5.3.1	Design Considerations	47
5.3.2	Object Reference and Data Type Resolution	48
5.3.3	Component Design	48
5.3.4	Error Handling	48
5.4	Retrieval System	49
5.4.1	Design Considerations	49
5.4.2	Vector Search Implementation	49
5.4.3	Query Generation	50
5.4.4	Query Templates	51
5.5	Agent	52
5.5.1	Design Considerations	53
5.5.2	State Management Architecture	53
5.5.3	Tool Management	54
5.5.4	Error Handling	55
5.6	Tool Design	55
5.6.1	Explorative Tools	55
5.6.2	Function Tools	56
5.6.3	Function Calling	56
6	Evaluation	57
6.0.1	Implementation Details	57
6.1	Model Selection	57
6.1.1	Requirements and Infrastructure	57
6.1.2	Selected Models	57
6.2	Evaluation Metrics	58
6.2.1	Metric Definitions and Calculations	58
6.3	Results	60
6.3.1	Quantitative	60
6.4	Discussion	61
6.4.1	Interpretation of Results	61
6.4.2	Implications	62

6.4.3 Future Research Directions	62
List of Figures	65
List of Tables	67
Listings	69
Bibliography	71
Eigenständigkeitserklärung	75

Kurzfassung

Diese Masterarbeit untersucht die Integration von Large Language Models (LLMs) in Spielentwicklungsumgebungen, mit besonderem Fokus auf ihre Anwendung als autonome Agenten in Unity. Die Forschung präsentiert einen neuartigen Ansatz zur Überbrückung der Lücke zwischen natürlichsprachlichem Verständnis und Spielsysteminteraktion durch die Entwicklung eines Proof-of-Concept-Agentensystems. Durch die Kombination von LLMs mit Wissensgraphen ermöglicht das System eine intuitive, sprachbasierte Steuerung von Spielobjekten bei gleichzeitiger Wahrung der für die Methodenausführung erforderlichen technischen Präzision.

Die Implementierung nutzt Wissensgraphen als semantische Datenbankrepräsentation von Unity-Spielszenen und ermöglicht eine effiziente Speicherung und Abfrage von hierarchischen Beziehungen und Spielzustandsinformationen. Dieser Ansatz ermöglicht eine kontextbewusste Methodenausführung und dynamische Zustandsverfolgung und adressiert damit zentrale Herausforderungen in der Spielumgebungsinteraktion. Das System demonstriert die Machbarkeit der Nutzung von LLMs für autonome Entscheidungsfindung im Spielkontext und beleuchtet dabei sowohl das Potenzial als auch die Grenzen aktueller Sprachmodell-Fähigkeiten.

Durch systematische Evaluation und Analyse trägt die Forschung zum Verständnis der LLM-Integration in Spielentwicklungsworkflows bei. Die Ergebnisse zeigen vielversprechende Richtungen für die natürlichsprachlich gesteuerte Spielentwicklung auf und identifizieren kritische Überlegungen für zukünftige Implementierungen. Diese Arbeit schafft eine Grundlage für intuitivere Interaktionsparadigmen in der Spielentwicklung und bietet Einblicke in die praktische Anwendung von Sprachmodellen in interaktiven Umgebungen.

Abstract

This thesis investigates the integration of Large Language Models (LLMs) with game development environments, specifically focusing on their application as autonomous agents within Unity. The research presents a novel approach to bridging the gap between natural language understanding and game system interaction through the development of a proof-of-concept agent system. By combining LLMs with knowledge graphs, the system enables intuitive, language-based control of game objects while maintaining the technical precision required for method execution.

The implementation leverages knowledge graphs as a semantic database representation of Unity game scenes, facilitating efficient storage and retrieval of hierarchical relationships and game state information. This approach enables context-aware method execution and dynamic state tracking, addressing key challenges in game environment interaction. The system demonstrates the feasibility of using LLMs for autonomous decision-making in

gaming contexts while highlighting both the potential and limitations of current language model capabilities.

Through systematic evaluation and analysis, the research contributes to the understanding of LLM integration in game development workflows. The findings reveal promising directions for natural language-driven game development while identifying critical considerations for future implementations. This work establishes a foundation for more intuitive interaction paradigms in game development, offering insights into the practical application of language models in interactive environments.

Chapter 1: Introduction

1.1 Background and Motivation

The landscape of artificial intelligence is undergoing a transformative shift with the continued advancement of Large Language Models (LLMs). These models have demonstrated remarkable capabilities showing consistent improvements in their ability to process and generate human-like text. This progression has catalyzed new possibilities in autonomous agent systems, where LLMs serve as cognitive cores capable of independent decision-making in complex problem-solving scenarios.

While LLMs excel at text generation and understanding, their integration with core systems remains a critical challenge. This limitation is particularly evident in interactive environments such as video games, where agents must bridge the gap between natural language understanding and direct system interaction. The gaming industry presents an ideal testing ground for such integration, offering complex, controlled environments where agents must make real-time decisions based on dynamic conditions.

1.2 Research Scope and Objectives

This thesis focuses on developing and evaluating a proof-of-concept agent system for interactive environments, specifically designed to execute methods within a Unity C# development environment through natural language input. The system aims to demonstrate how LLMs can be effectively integrated with game environments to enable intuitive, language-based control of game objects and systems.

The primary objectives of this research are:

1. Develop an agent system capable of understanding and processing natural language input from players
2. Enable accurate identification and execution of relevant methods and actors within the Unity environment
3. Implement autonomous decomposition of complex, multi-step requests into executable sub-tasks
4. Ensure proper handling of C# programming principles, including:

- Primitive and reference data types
- Object-oriented concepts (inheritance, polymorphism)
- Unity-specific programming patterns

1.3 Technical Innovation

A key innovation in this research is the integration of knowledge graphs as a semantic database representation of the Unity game scene. Knowledge graphs are particularly well-suited for representing the hierarchical and referential relationships common in Unity environments, enabling:

- Context-aware method retrieval and execution
- Efficient representation of scene hierarchies and object relationships
- Dynamic updating of game state information
- Semantic search capabilities for natural language queries

This approach combines the semantic understanding capabilities of LLMs with structured knowledge representation, providing a robust foundation for intelligent agent behavior in game environments.

1.4 Research Methodology

The research follows a design-oriented approach, combining theoretical analysis with practical implementation and evaluation. The methodology encompasses:

1. Comprehensive literature review covering LLMs, agent architectures, and knowledge graph integration
2. Analysis of existing game-oriented agent systems to identify effective patterns and current limitations
3. Iterative development and refinement of the agent system components
4. Experimental evaluation using designed test scenarios
5. Critical analysis of results and identification of future research directions

1.5 Scope Limitations

To maintain focus and feasibility, the following limitations have been established:

- The primary scope centers on accurate function identification and execution
- Role distribution and access control are not explicitly considered
- Security aspects such as jailbreaking attempts are intentionally excluded
- The implementation focuses on Unity-specific programming patterns and constraints

1.6 Thesis Structure

The remainder of this thesis is organized as follows:

- Chapter 2 presents a comprehensive literature analysis of relevant technologies and theoretical frameworks
- Chapter 3 outlines the research methodology and development approach
- Chapter 4 examines related works, analyzing contemporary agent systems in gaming contexts
- Chapter 5 details the system design and architectural decisions and implementation details and technical specifications
- Chapter 6 presents system testing and evaluation results and concludes with findings, limitations, and future research directions

1.7 Expected Contributions

This research aims to advance the field of natural language-driven game development by demonstrating the feasibility of LLM-powered agent systems in Unity environments. The integration of knowledge graphs for scene representation offers a novel approach to context management in game development workflows. Specific contributions include:

1. Demonstrating the feasibility of LLM-powered agent systems in Unity environments
2. Establishing effective patterns for integrating LLMs with game development workflows
3. Providing novel approaches to knowledge graph integration for context management
4. Identifying key challenges and solutions in bridging natural language understanding with game system interaction

The findings from this research will contribute to both the theoretical understanding of LLM-based agents and practical implementations in game development contexts, paving the way for more intuitive and powerful interaction paradigms in interactive environments.

Chapter 2: Background and Literature Review

This chapter provides a comprehensive overview of the theoretical foundations and relevant literature that underpin both the conceptual framework and practical implementation of the proof of concept. While some of the topics discussed have been subjects of research for decades, others have emerged only recently, particularly in the rapidly evolving field of large language models. Much of the information presented draws from recent survey papers that examine the current state of the art. Given the dynamic nature of LLM research, with new findings emerging frequently, this chapter aims to present the most current understanding of these topics while acknowledging the field’s rapid evolution.

2.1 Large Language Models

2.1.1 Understanding Language Models

Large language models represent a fundamental technology in modern artificial intelligence, enabling machines to understand, generate, and manipulate human language. As Jurafsky and Martin (2024, p. 204) explain, these models process text by predicting the probability of words or tokens in a sequence, having learned these patterns from vast amounts of training data.

The concept of language models has evolved significantly from simple statistical approaches to sophisticated neural architectures. A pivotal advancement came with the introduction of the Transformer architecture by Vaswani et al. (2023), which revolutionized natural language processing through its self-attention mechanism. This mechanism enables the model to weigh the importance of different words when interpreting any given token, allowing it to capture both local and long-distance relationships in text (Jurafsky & Martin, 2024, p. 185). Unlike previous architectures that processed text sequentially, Transformers can process all input tokens simultaneously, dramatically improving both performance and the model’s ability to capture complex language patterns.

Modern large language models acquire their capabilities through a process called pretraining—learning knowledge about language and the world from vast amounts of text. As Jurafsky

and Martin (2024, p. 204-205) describe, through this process, they develop rich representations of language patterns, semantic relationships, and world knowledge. This pretraining enables them to perform what they term "conditional generation"—the task of generating text based on an input piece of text (called a prompt) while maintaining coherence and relevance throughout long sequences.

2.1.2 Core Capabilities and Limitations

According to Matarazzo and Torlone (2025, p. 7), LLMs demonstrate two distinct categories of capabilities: fundamental capabilities that encompass essential text processing functions, and emergent capabilities that arise from quantitative system improvements.

The fundamental capabilities include:

- **Text generation:** LLMs excel at producing coherent and contextually appropriate content across various domains
- **Question-answering:** They provide relevant responses by leveraging their trained knowledge base
- **Language translation:** They enable text conversion between different languages while maintaining semantic meaning
- **Summarization:** They effectively condense longer texts while preserving crucial information
- **Sentiment analysis:** They accurately determine emotional tone and attitude in text

Beyond these foundational abilities, LLMs exhibit three significant emergent capabilities (Matarazzo & Torlone, 2025, p. 8):

- **In-context learning:** Models generate appropriate outputs based on natural language instructions without additional training, adapting to new tasks by understanding examples and patterns within the prompt itself
- **Instruction following:** Models perform new tasks through natural language descriptions alone, understanding and executing instructions without explicit programming
- **Step-by-step reasoning:** Models handle complex tasks requiring multiple cognitive steps through chain-of-thought processes, decomposing problems into manageable components

This versatility stems from the principle that "many practical NLP tasks can be cast as word prediction" (Jurafsky & Martin, 2024, p. 204).

Limitations

Despite these capabilities, LLMs face several significant limitations:

- **Hallucination and Factual Accuracy:** LLMs tend to generate plausible but factually incorrect information, a phenomenon known as "hallucination" (Jurafsky & Martin, 2024, p. 206). This stems from their training optimization for predictable and coherent text generation without inherent fact-verification mechanisms.
- **Reasoning and Inference:** It is highly debated whether llms are capable of true reasoning. Studies suggest that LLMs primarily engage in probabilistic pattern-matching rather than formal logical reasoning (Jiang et al., 2024). They often generate responses based on statistical associations, leading to inconsistent outputs when introducing minor input variations (Jiang et al., 2024; Shi et al., 2023). Mirzadeh et al. (2024) demonstrated that adding inconsequential details to questions significantly reduces model performance.
- **Planning and Self-Verification:** While LLMs can mimic planning through pattern recognition, they struggle with independently planning and self-verifying solutions. This limitation becomes particularly evident in tasks requiring structured reasoning or subgoal resolution. Attempts at "self-critiquing" methods have proven ineffective according (Matarazzo & Torlone, 2025).
- **Text Generation Process:** LLMs can fall into patterns of repetitive token generation, requiring external control parameters such as stopping tokens or maximum length constraints. This suggests they may not truly understand context or reason about appropriate response structure (Matarazzo & Torlone, 2025).

2.2 LLMs as Agents

Recent research demonstrates that LLMs can serve as effective cognitive cores for autonomous agents, enhancing their planning and decision-making capabilities (Huang et al., 2024b). The broad knowledge base acquired through pretraining enables these models to execute actions without task-specific training (Wang et al., 2024), while their ability to process and generate human-like text facilitates natural interaction with both users and structured knowledge sources.

2.2.1 Agent Fundamentals

An agent, in its fundamental form, represents a system that operates within a defined environment and performs goal-directed actions (Wooldridge & Jennings, 1995). This foundational concept provides the basis for developing more sophisticated agent architectures. The effective integration of LLMs into agent systems requires careful consideration of architectural design and the interfaces between the LLM and other system components.

2.2.2 Architectural Framework

A recent review paper by Wang et al. (2024) examined the growth in LLM-based autonomous agents between January 2021 and August 2023, proposing a comprehensive framework for their development. Their analysis identifies four essential modules that form the foundation of contemporary LLM-based agent architectures: profiling, memory, planning, and action. Each module serves distinct yet interconnected functions that collectively enable sophisticated autonomous behavior.

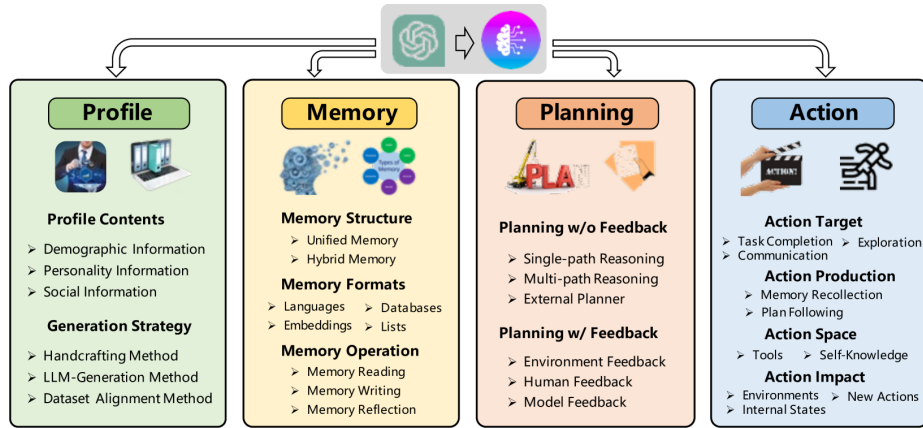


Figure 2.1: "A unified framework for the architecture design of LLM-based autonomous agent." Source: Wang et al., 2024

The following is a summary of the architectural framework as proposed by Wang et al. (2024).

Profiling Module

The profiling module establishes the foundation for agent identity and behavioral patterns. According to Wang et al. (2024), autonomous agents typically operate by assuming specific roles, such as programmers, educators, or domain experts. The module defines these roles through several key components:

- **Demographic Information:** Core characteristics that define the agent's fundamental attributes and operational context
- **Personality Traits:** Behavioral and psychological characteristics that shape the agent's response patterns and decision-making processes
- **Social Information:** Contextual data about the agent's relationships and position within its operational environment

These elements collectively influence the agent's decision-making processes and interaction patterns within its operational environment.

Memory Module

The memory architecture implements sophisticated mechanisms for information processing and retention, drawing parallels with human cognitive processes. The module supports two primary memory structures:

Memory Structures

- **Unified Memory:** Implements short-term memory through in-context learning, where memory information is directly incorporated into prompts. While straightforward, this approach faces limitations due to LLMs' context window constraints.
- **Hybrid Memory:** Models both short-term and long-term memory components explicitly. Short-term memory buffers current perceptions, while long-term memory consolidates important information over time. This architecture enhances the agent's ability to reason across extended periods and accumulate valuable experiences.

Memory Formats The implementation of memory systems can take several forms:

- **Natural Language Storage:** Maintains memory through raw natural language descriptions, preserving semantic richness while enabling flexible interpretation.
- **Embedding Storage:** Encodes memory information as vectors, facilitating efficient retrieval and processing operations.
- **Database Storage:** Organizes memory in structured databases, enabling systematic management and sophisticated query capabilities.

Planning Module

The planning module enables agents to decompose complex tasks into manageable subtasks and develop structured approaches to problem-solving. This capability simulates human cognitive abilities in forward thinking and action sequencing, contributing to more rational and reliable agent behavior.

Planning Strategies Planning strategies in LLM-based agents fall into two main categories:

1. Feedback-free Planning:

- **Single-Path Reasoning:** Decomposes tasks into linear sequences, employing techniques like Chain of Thought (CoT) and Zero-shot CoT
- **Multi-Path Reasoning:** Organizes thinking in tree structures, allowing consideration of multiple possibilities through methods like Tree of Thoughts (ToT)
- **External Planner Integration:** Utilizes domain-specific planning tools through formal Planning Domain Definition Languages (PDDL)

2. Feedback-based Planning:

- **Environmental Feedback:** Incorporates signals from the objective world or virtual environment
- **Human Feedback:** Integrates human interaction to enhance planning and reduce hallucination
- **Model Feedback:** Utilizes internal feedback mechanisms for self-refinement and reflection

Action Module

The action module serves as the interface between the agent's decision-making processes and its environment, translating abstract plans into concrete outcomes. This module integrates closely with the profiling, memory, and planning components to influence the agent's environmental interactions.

Action Objectives The module pursues several key objectives:

- **Task Completion:** Execution of specific functions and achievement of defined goals
- **Communication:** Facilitation of information exchange with other agents or humans
- **Environment Exploration:** Investigation of unknown environments while balancing exploration and exploitation

Action Production Actions can be generated through two primary mechanisms:

- **Memory-Based Generation:** Actions triggered by retrieving relevant past experiences
- **Plan-Based Generation:** Actions executed according to previously formulated plans

Action Space The action space encompasses all possible actions an agent can execute:

- **External Tools:**
 - APIs for accessing external resources
 - Domain-specific knowledge bases
 - Specialized models for complex tasks
- **Internal Capabilities:**
 - Planning and task decomposition
 - Natural language interaction
 - Common sense reasoning

Action Impact Actions can affect multiple aspects of the system:

- **Environmental Changes:** Physical or virtual environment modifications
- **Internal State Updates:** Modifications to the agent’s knowledge or memory
- **Action Chains:** Initiation of subsequent actions for task completion

The integration of these four modules—profiling, memory, planning, and action—creates a comprehensive framework for LLM-based agent systems. However, successful implementation requires robust knowledge integration mechanisms and sophisticated planning frameworks, which we will explore in subsequent sections. The memory module, in particular, highlights the need for structured knowledge representation and efficient retrieval systems, leading us to examine Retrieval-Augmented Generation (RAG) and knowledge graph integration in the following section.

2.3 Enhancing Memory through Knowledge Integration

The effective implementation of LLM-based agent systems necessitates sophisticated approaches to knowledge integration. While LLMs possess extensive internal knowledge from pretraining, their limitations in factual accuracy and knowledge staleness create a compelling need for external, updatable knowledge sources. This section examines two key technologies that address these challenges: Retrieval-Augmented Generation (RAG) and knowledge graphs. These technologies complement each other, with RAG providing dynamic information retrieval capabilities and knowledge graphs offering structured representation of relationships and concepts.

2.3.1 Retrieval-Augmented Generation

Retrieval-Augmented Generation (RAG) represents a significant advancement in Artificial Intelligence Generated Content (AIGC) (Zhao et al., 2024), addressing key challenges faced by traditional generative models such as outdated knowledge, handling rare or specialized information (long-tail data) (Mallen et al., 2023) and high computational costs during training and inference (Izacard et al., 2022; Wu et al., 2022; He et al., 2024).

Core Architecture

RAG systems integrate information retrieval directly into the generation process, enabling models to access external data sources in real-time. This integration serves as a dynamic and updatable memory system (Kang et al., 2024), enhancing the accuracy and relevance of generated content by incorporating current and context-specific information.

According to Zhao et al. (2024, Chapter II-A), a typical RAG system comprises two primary modules:

1. **Retriever:** Searches for relevant information from the data store based on input queries
2. **Generator:** Produces content using both the original query and retrieved information

The RAG process follows a three-stage workflow:

- (i) The retriever initially receives the input query and searches for relevant information;
- (ii) The original query and the retrieval results are fed into the generator through a specific augmentation methodology;
- (iii) Finally, the generator produces the desired outcomes.

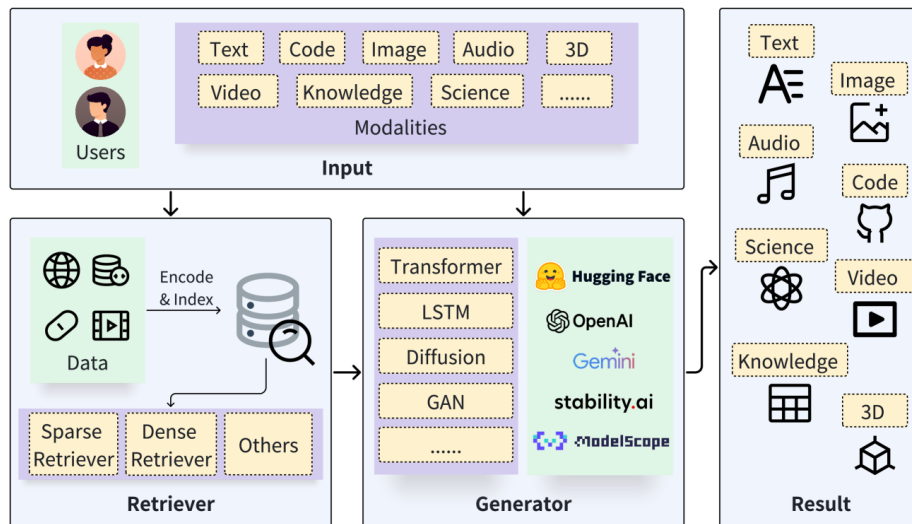


Figure 2.2: "A generic RAG architecture. The user queries, spanning different modalities, serve as input to both the retriever and the generator. The retriever extracts relevant information from data sources. The generator interacts with the retrieval results and ultimately produces outcomes of various modalities." Source: Zhao et al., 2024

Information Integration Paradigms

Zhao et al., 2024 define in their survey paper "Retrieval-Augmented Generation for AI-Generated Content: A Survey" four fundamental paradigms that describe how retrieved information can be integrated into generative AI models.

1. Query-based RAG

In this method, also known as prompt augmentation, the retrieved information is directly combined with the user query and passed to the generator as input. The integration of the retrieved data can be done through simple concatenation or through

more complex prompt engineering techniques. The goal is to expand the context of the query and provide the generator with additional information.

2. Latent Representation-based RAG

Unlike query-based RAG, in latent representation-based RAG the retrieved information is not directly fed into the input but is integrated into the generator as latent representations. These representations are typically created by separate encoders and then combined with the representations of the content to be generated at a deeper level. This approach allows for a more subtle and complex interaction between the retrieved data and the generation process.

3. Logit-based RAG

Logit-based RAG influences the probability distribution of generation during the decoding process by integrating retrieved information through logits. Logits represent the unnormalized probabilities for individual tokens to be generated next. Retrieved data is used to modify the logits, thereby influencing the selection of the most probable tokens in the next step.

4. Speculative RAG

Speculative RAG methods attempt to use retrieval instead of pure generation to save resources and increase response speed. Instead of generating each token individually, pre-existing text snippets or code fragments from the retrieved data are used directly.

These four RAG Foundations offer a wide range of possibilities for integrating retrieved information into generative AI models. The choice of the appropriate paradigm depends on the specific application and the requirements for the quality and efficiency of generation.

2.3.2 Knowledge Graphs as Structured Memory

The integration of graph databases with Retrieval-Augmented Generation (RAG) systems represents a critical advancement in agent architecture, particularly in maintaining and querying environmental knowledge. Recent research demonstrates significant progress in query generation and natural language integration, supported by comprehensive experimental validation.

Fundamentals of Knowledge Graphs

Knowledge graphs have emerged as a central concept in data processing and knowledge management over the past decade and gained renewed significance with Google's introduction of their Knowledge Graph in 2012. Today, knowledge graphs are utilized by numerous enterprises and research institutions to access and manage large, dynamic collections of data. Hogan et al. (2021, Chapter 1)

2.3.3 Basic Concepts and Definition

According to Serles and Fensel (2024, p. 65) “A knowledge graph is a large semantic net that integrates heterogeneous data sources into a graph structure. Instead of providing a large set of rules for deducing answers to questions, it contains many explicit facts, which is its main difference from traditional knowledge-based systems. Finally, a graph is a mathematical structure in which some pairs in a set of objects are somehow related.”

Jurafsky and Martin (2024, p. 2) define a knowledge graph as “[...] a graph of data intended to accumulate and convey knowledge of the real world, whose nodes represent entities of interest and whose edges represent relations between these entities.”

So at its core, a knowledge graph is a data structure that aims to accumulate and convey knowledge about the real world through a graph-based representation.

Graphs are inherently intuitive, as they can represent complex relationships between entities through edges and paths. They allow for flexible data evolution, as a schema need not be strictly defined in advance. Hogan et al. (2021, Chapter 1). Furthermore, graph query languages support navigation operators to identify entities connected through paths of arbitrary length.

2.3.4 Graph Structures and Models

Hogan et al. (2021) list various data models as to how a knowledge graph might be represented, each with distinct characteristics and capabilities:

Directed Edge-Labeled Graphs (DEL Graphs)

DEL graphs represent the most fundamental graph model employed in knowledge graphs. They consist of a set of nodes representing entities and a set of directed, labeled edges representing binary relationships between these entities. Hogan et al., 2021, Chapter 2.1.1



Figure 2.3: "Directed-edge labelled graph describing events and their venues." Source: Hogan et al., 2021

DEL graphs offer significant flexibility in integrating new data sources, as they do not require a rigid schema definition.

Heterogeneous Graphs

Heterogeneous graphs extend the DEL graph model by assigning types to both nodes and edges. These types form an integral part of the graph model itself, rather than being expressed as special relations as in DEL graphs. The model distinguishes between homogeneous edges (connecting nodes of the same type) and heterogeneous edges (connecting nodes of different types). Hogan et al., 2021, Chapter 2.1.2



Figure 2.4: "Data about capitals and countries in a del graph and a heterogeneous graph." Source: Hogan et al., 2021

While heterogeneous graphs facilitate node partitioning by type they typically assume a one-to-one relationship between nodes and types, limiting nodes to a single type assignment. This contrasts with DEL graphs, where nodes can possess multiple types.

Property Graphs

Property graphs enhance the basic graph model by allowing nodes and edges to carry attribute-value pairs and labels. This extension provides greater flexibility in data modeling compared to simpler graph models. While offering intuitive data representation, particularly for complex attribute modeling, this increased flexibility necessitates more sophisticated query languages, formal semantics, and inductive techniques. Hogan et al., 2021, Chapter 2.1.3

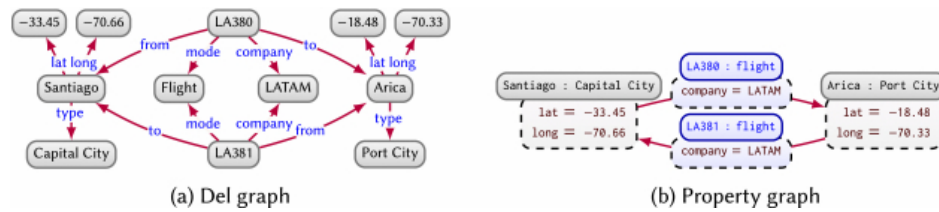


Figure 2.5: "Flight data in a del graph and a property graph." Source: Hogan et al., 2021

2.3.5 Data Retrieval

The ability to effectively query knowledge graphs is fundamental to utilizing their stored information. Various languages and techniques have been developed for graph querying,

with the selection of appropriate methods depending on the underlying data model, specific application requirements, and query complexity.

Graph Patterns

Graph patterns form the foundation of knowledge graph querying. A graph pattern essentially represents a small graph that corresponds to the data model of the knowledge graph being queried, but incorporates variables to enable pattern matching within the data graph. Hogan et al., 2021, Chapter 2.2.1

Two primary semantic approaches govern pattern evaluation:

- Homomorphism-based semantics, permitting multiple variables to map to the same term (implemented in SPARQL)

```
1 _:paper1 rdf:type :Publication ;
2         :title "Semantic Web Technologies" ;
3         :author _:author1 .
4
5 _:author1 :name "Jane Smith" ;
6         :affiliation "University of Technology" .
7
8 _:paper2 rdf:type :Publication ;
9         :title "SPARQL Query Optimization" ;
10        :author _:author2 .
11
12 _:author2 :name "John Doe" .
```

The SPARQL query demonstrating homomorphism-based semantics:

```
1 SELECT ?title ?authorName ?affiliation
2 WHERE {
3   ?paper rdf:type :Publication ;
4         :title ?title ;
5         :author ?author .
6   ?author :name ?authorName .
7   OPTIONAL {
8     ?author :affiliation ?affiliation .
9   }
10 }
```

- Isomorphism-based semantics, requiring unique mappings between variables and nodes or edges (used in Cypher for edge variables)

```
1 CREATE
2 (p1:Publication {title: "Semantic Web Technologies"}),
3 (p2:Publication {title: "Graph Query Optimization"}),
4 (a1:Author {name: "Jane Smith",
```



```

5         affiliation: "University of Technology"}),
6 (a2:Author {name: "John Doe"}),
7 (p1)-[:AUTHORED_BY]->(a1),
8 (p2)-[:AUTHORED_BY]->(a2)

```

The Cypher query demonstrating isomorphism-based semantics:

```

1 MATCH (p:Publication)-[:AUTHORED_BY]->(a:Author)
2 OPTIONAL MATCH (a)-[r:AFFILIATED_WITH]->(i:Institution)
3 RETURN p.title as title,
4         a.name as authorName,
5         a.affiliation as affiliation

```

Complex Graph Patterns

Complex graph patterns extend basic pattern matching by enabling the transformation of tabular results through relational algebra operations. These operations include:

- Projection (π , SELECT)
- Selection (σ , WHERE or FILTER)
- Union ($\cup$, UNION)
- Difference ($-$, EXCEPT)
- Inner join ($\bowtie$, NATURAL JOIN)
- Left outer join ($\bowtie\leftarrow$, LEFT OUTER JOIN or OPTIONAL)
- Anti-join ($\bowtie\neg$, NOT EXISTS)

Complex patterns may generate duplicate results, leading query languages to distinguish between bag semantics (preserving duplicates according to their multiplicity) and set semantics (eliminating duplicates through DISTINCT operations). Hogan et al., 2021, Chapter 2.2.2

Navigational Patterns

Navigational patterns enable the definition and querying of paths within a graph, particularly valuable when investigating relationships spanning multiple edges. Path expressions, denoted as r , constitute regular expressions used in queries of the form (x, r, y) , where x and y represent variables or constants. Hogan et al., 2021, Chapter 2.2.3 The fundamental path expression components include:

- r^{-1} (inverse)
- r^* (zero or more occurrences)
- $r_1 \mid r_2$ (disjunction)
- $r_1 \cdot r_2$ (concatenation)

The evaluation of path expressions can employ various semantics, particularly when handling cycles that could generate infinite paths. Common approaches include restricting results to shortest paths or paths without repeated nodes, or returning the finite set of node pairs connected by matching paths. Hogan et al., 2021, Chapter 2.2.3

Retrieval Techniques

Xu et al., 2024 categorizes Knowledge Graph Retrieval into three main approaches:

1. **Retrieval-based** These approaches leverage relation extraction and distributed representations to retrieve answers from Knowledge Graphs (KGs) (Xu et al., 2016). While effective for single-query scenarios, they face limitations in multi-query contexts that require comprehensive KG understanding (Bordes et al., 2014). Distributed representations help capture semantic relationships between entities for question-answering systems.
2. **Template-based** Template-based methods employ predefined patterns to convert complex queries into structured formats for KG navigation. Unger et al., 2012 developed a parser that transforms natural language inputs into SPARQL templates with URI-fillable slots, enabling more flexible query formulation. However, the effectiveness of this approach is limited by the availability and coverage of predefined templates.
3. **Semantic Parsing-based** These methods convert natural language queries into executable logical forms for knowledge graphs. A recent study by Emonet et al., 2024 developed a Retrieval-Augmented Generation (RAG) system that utilizes bioinformatics KG metadata to translate natural language questions into SPARQL queries using LLMs. Their evaluation revealed that while larger models like GPT-4 achieved impressive results (F1-score: 0.91) with RAG and validation, smaller models (GPT-4-mini, Mixtral 8x22B, and Llama3.1 8B) performed significantly worse, with F1-scores of 0.37, 0.33, and 0.08 respectively.

2.4 Planning and Action Frameworks

With enhanced memory and knowledge integration capabilities established, agent systems require robust mechanisms for planning and executing actions. This section explores how LLMs can be effectively used for reasoning and planning, as well as how they can be integrated with external tools and APIs for action execution.

LLMs demonstrate significant potential for complex reasoning and planning tasks when properly prompted.

2.4.1 Prompt Engineering

Prompt engineering has emerged as a crucial discipline in the effective utilization of LLMs. It encompasses a range of techniques and methodologies for crafting input prompts that elicit desired behaviors and outputs from these models. The field has evolved from simple instruction-based approaches to sophisticated frameworks that enable complex reasoning and task execution.

Basic Prompting Techniques

Zero-Shot Prompting

Zero-shot learning represents the ability of LLMs to perform tasks without any examples or demonstrations in the prompt Kojima et al., 2023. This capability emerges from extensive pre-training on diverse datasets, enabling models to understand and execute instructions directly (in-context learning).

The significance of zero-shot learning lies in its efficiency and versatility. It demonstrates the model's ability to generalize from its pre-training to new, unseen tasks without requiring task-specific examples or fine-tuning. This capability is particularly valuable in scenarios where example data is scarce or when rapid deployment across diverse tasks is needed.

Few-Shot Prompting

Few-shot learning extends the model's capabilities by providing demonstrations within the prompt, as outlined by Brown et al. (2020). This technique becomes particularly valuable for complex tasks where zero-shot performance may be insufficient.

Few-shot prompting typically includes:

- **Task Description:** Clear explanation of the desired outcome.
- **Examples:** One or more demonstrations of the task.
- **Target Instance:** The specific case requiring a solution.

The number of examples can vary (e.g., 1-shot, 3-shot, 5-shot) depending on task complexity and required performance. Brown et al. (2020) argue that few-shot prompting dramatically increases performance across all domains, as the context of the prompt is the main contributor to generating output tokens.

However, few-shot prompting alone may not suffice for tasks requiring complex logical steps and is also very domain-specific, needing specific examples for each problem, which cannot be generalized easily. Another downside is that with increasing example count, the input token count also increases, which translates to higher costs and can, depending on the LLM, reach the context window limit.

2.4.2 Reasoning frameworks

Chain-of-Thought (CoT) Prompting CoT prompting, first introduced by Wei et al. (2023) integrates intermediate reasoning steps within prompts, providing a structured path toward solutions.

Template

"Let's think step by step."

Wei et al., 2023, p. 2

Example

Prompt:

Let's solve this math problem step by step:

In a bookstore, Sarah bought 3 novels at \$12 each and 2 magazines at \$5 each.

If she paid with a \$50 bill, how much change did she receive?

Model Response:

Let me break this down:

1. Calculate cost of novels:
 $3 \text{ novels} \times \$12 = \$36$
2. Calculate cost of magazines:
 $2 \text{ magazines} \times \$5 = \$10$
3. Calculate total purchase amount:
 $\$36 + \$10 = \$46$
4. Calculate change from \$50:
 $\$50 - \$46 = \$4$

Therefore, Sarah received \$4 in change.

ReAct (Reasoning+Acting) ReAct is implemented as an augmented action space approach that combines reasoning and acting:

- **Action Space Augmentation:** Extends the traditional action space to include language-based reasoning, allowing agents to generate explanatory thoughts alongside actions.
- **Thought Integration:** Incorporates free-form language thoughts that reason over context without affecting the environment, supporting future decisions.

Yao et al., 2023, p. 3 **Template**

Question: [question text]
 Thought 1: Let me approach this step by step
 Action 1: [First action to take]
 Observation 1: [Result of the action]
 Thought 2: [Analysis of the observation and planning next step]
 Action 2: [Next action to take]
 Observation 2: [Result of the action]
 ...
 Therefore, the answer is [final answer]

Example

Question: In a bookstore, Sarah bought 3 novels at \$12 each and 2 magazines at \$5 each.
 If she paid with a \$50 bill, how much change did she receive?
 Thought 1: Let me calculate this step by step. First, I need to find the cost of the novels.
 Action 1: Calculate cost of novels: 3 \$12
 Observation 1: The novels cost \$36 total
 Thought 2: Now I need to calculate the cost of the magazines
 Action 2: Calculate cost of magazines: 2 \$5
 Observation 2: The magazines cost \$10 total
 Thought 3: I can now find the total cost and subtract from \$50
 Action 3: Calculate total cost (\$36 + \$10) and subtract from \$50
 Observation 3: Total cost is \$46, change is $50 - 46 = 4$
 Therefore, the answer is \$4 in change.

The combination of structured reasoning frameworks and well-integrated tools creates a powerful foundation for agent systems. By leveraging LLMs for planning while connecting them to concrete actions through tools and APIs, agents can effectively operate in complex environments while maintaining reliability and performance.

Chapter 3: Methodology

3.1 Research approach

This prototype investigates how natural language inputs can be utilized to control agent systems, particularly in the form of Non-Player Characters (NPCs). The primary objective is to determine whether an agent system can autonomously find solutions to user-specified problems using pre-programmed methods and metadata—such as descriptions, method names, and parameter lists—without extensive instructions or techniques like few-shot prompting.

Additionally, the research examines how Knowledge Graphs can be employed to store and efficiently retrieve relevant real-time game data, such as scene objects and their references to other in-game variables, enabling informed decision-making within the game context. Furthermore, it explores how Large Language Models (LLMs) can be leveraged to support autonomous decision-making processes.

The methodological approach is based on Design Research, which generates new knowledge through iterative development and evaluation of artifacts. The prototype undergoes multiple revisions and is evaluated against specific criteria to ensure continuous improvement. This approach was chosen due to the limited existing research on integrating these technologies into agent systems, necessitating an innovative, practice-oriented development process.

Through exploratory prototyping phases and experimental testing, fundamental connections and potential synergies between Knowledge Graphs and LLMs are identified. This exploratory element is incorporated to gain a deeper understanding of the relationships between the systems before conducting detailed analyses and refinements.

3.2 Implementation process

The prototype’s development process was divided into several phases to ensure structured and efficient implementation:

1. **Literature Review** A comprehensive literature review was conducted covering:
 - **Large Language Models (LLMs)**
 - **Retrieval-Augmented Generation (RAG)**

- **RAG with Knowledge Graphs**
- **Agent-based Workflows**

The detailed findings are presented in Chapters 2 and 3. The analysis revealed specific strengths and weaknesses of each system and significantly influenced the prototype's implementation requirements. It became evident that the combination of these technologies within Unity and other environments remains largely unexplored or only partially implemented.

2. **Concept Development** This phase involved analyzing three relevant agent systems in gaming and interactive system contexts, examining their agent frameworks, evaluation methods, results, and key findings. This analysis provides an overview of current achievements and limitations likely to be encountered in our prototype, forming the foundation for our agent design concept.
3. **Module Development**
 - **Knowledge Graph Modeling** A knowledge graph was modeled to store structured real-time game data, including scene objects and hierarchies, enabling efficient information management and querying.
 - **Retrieval Mechanism Implementation** A mechanism was developed for efficiently retrieving relevant information from the knowledge graph and executing corresponding function calls, ensuring the agent's access to current game data.
 - **LLM Integration and Agent Graph Modeling** The Large Language Model was integrated to process natural language inputs and support the agent system's decision-making. An agent graph was modeled to represent possible actions and states of the agent.
 - **Tool Modeling and Design** Following the integration of core modules, specialized tools and interfaces were developed to facilitate component interaction, including interfaces, user interfaces, and functions streamlining communication between the knowledge graph, LLM, and development environment.
4. **Experimental Investigation** Module performance was evaluated through qualitative experiments using designed test scenarios to verify functionality and effectiveness. These experiments provided valuable insights into the prototype's strengths and limitations.
5. **Analysis, Refinement, and Further Development** Based on experimental findings, iterative improvements were made to individual modules. This analysis and refinement process led to further module development in an iterative cycle, achieving optimal synergy between modules and enhancing overall prototype performance.

The iterative nature of this development process enabled continuous module optimization and adaptation to new insights. Specific details of the implementation process steps are elaborated in subsequent chapters.

Chapter 4: Analysis

This chapter analyzes contemporary LLM-based game agents to inform the development of our own agent architecture. Through systematic examination of existing implementations, their evaluation methodologies, and key findings, we identify successful patterns and current limitations in the field. This analysis provides the foundation for our proposed architecture, which aims to address identified challenges while building upon proven approaches.

4.1 Analysis of Contemporary Solutions

The integration of Large Language Models (LLMs) as agents reveals interesting developments in artificial intelligence for games. Recent research demonstrates advances in agent control and interaction within complex game environments. This section presents a comparative analysis of select projects, focusing on their architectures, evaluation methodologies, and limitations, to inform our own implementation.

Our analysis examines three contemporary game agent implementations that represent distinct approaches to LLM-based agent design. These systems were selected based on their:

- Recent development (all published within the last two years)
- Focus on game environments
- Diverse approaches to agent architecture

Jeurissen et al. (2024) present NetPlay, an agent designed to navigate the complex roguelike game 'NetHack' using Large Language Models (LLMs). Their study, "Playing NetHack with LLMs: Potential & Limitations as Zero-Shot Agents," investigates the capabilities and constraints of LLMs as zero-shot agents in complex game environments. The architecture leverages predefined capabilities and dynamic context information to handle NetHack's high complexity and unpredictability.

Huang et al. (2024a) introduce WESE (Weak Exploration to Strong Exploitation), an innovative approach to enhance LLM agents in interactive open-world tasks. Their paper, "WESE: Weak Exploration to Strong Exploitation for LLM Agents," describes a system that decouples exploration and exploitation processes. WESE employs a cost-effective LLM agent for exploration, storing discovered knowledge in a knowledge graph, which a more powerful LLM agent then utilizes for efficient problem-solving.

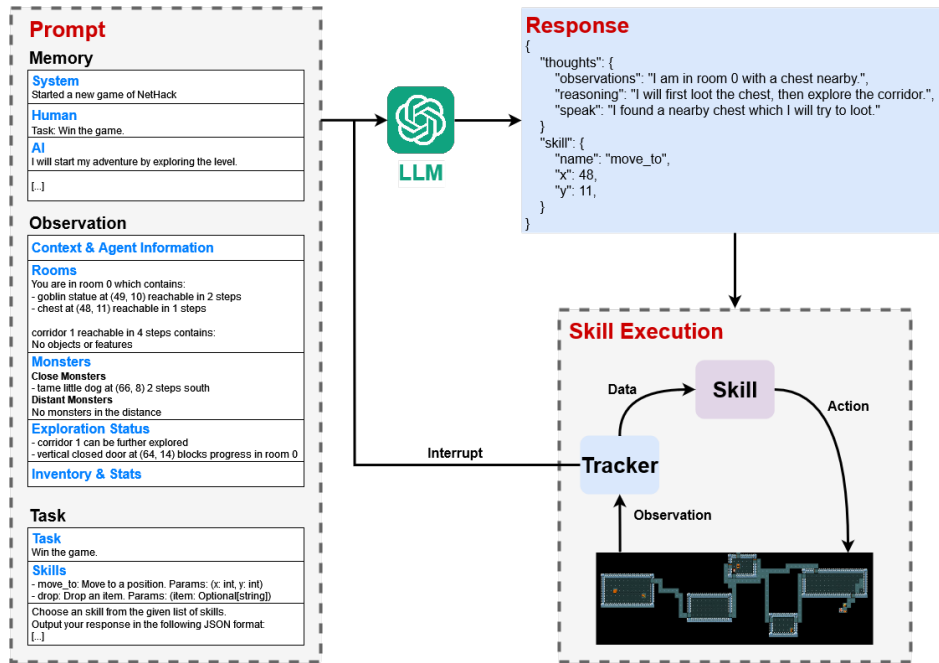


Figure 4.1: "Illustration of NetPlay playing NetHack. The process involves constructing a prompt using messages representing past events, the current observation, and a task description containing available skills and the desired output format. The response is parsed to retrieve the next skill. While executing the selected skill, a tracker enriches the given observations and detects important events, such as when a new monster appears. When the skill is done, or events interrupt the skill execution, the agent will restart the prompting process." Source: Jeurissen et al., 2024, Figure 2

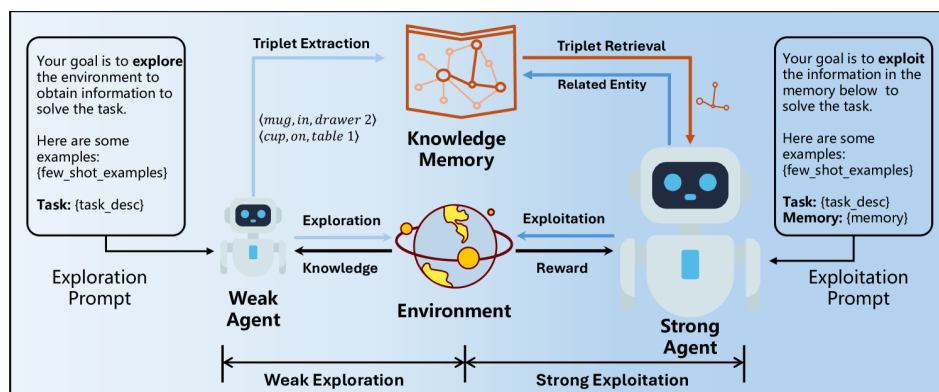


Figure 4.2: "Framework of WESE. The left part represents the weak exploration and the right part represents the strong exploitation. We employ Llama-2-7B as the weak agent and text-davinci-003 as the strong agent in the implementation." Source: Huang et al., 2024a, Figure 2

Sanchez Llado (2024)'s Master's thesis, "Controlling Agents Behaviours through LLMs" (referred to as CAB), develops an agent system for navigating maze-like environments and solving puzzles. The research evaluates various LLM architectures and configurations in both a simplified maze and the more complex "Obstacle Tower" environment.

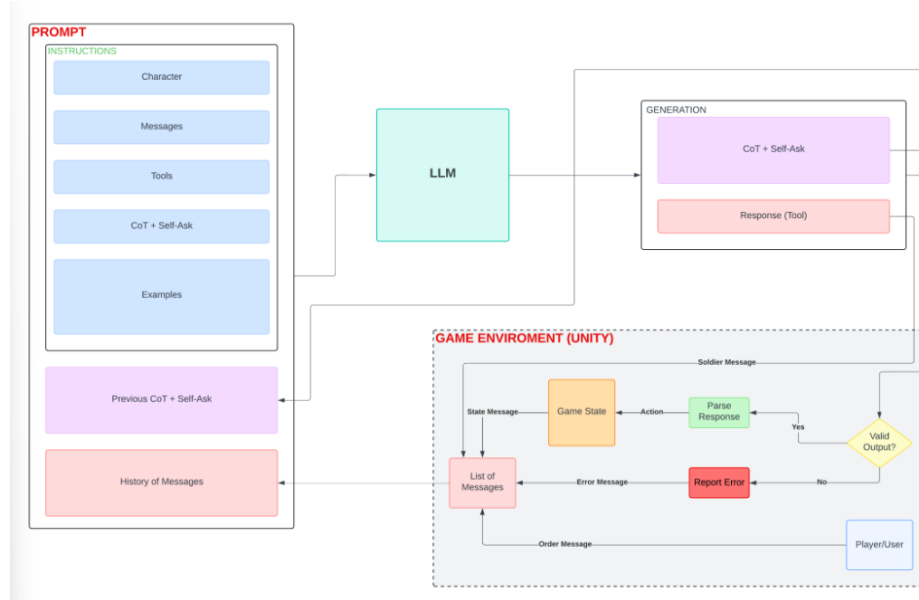


Figure 4.3: "General architecture of the LLM agent. The agent in the game environment requests the LLM for an action through a prompt. Then the agent attempts to parse the response, and after attempting the action, it starts the next requests." Source: Sanchez Llado, 2024, Figure 3.4

4.2 Implementation Analysis

4.2.1 Agent Framework Implementations

Building upon the fundamental components established in Chapter 2, we analyze these modern agent architectures through four essential modules: profiling, memory, planning, and action. This framework enables systematic evaluation of different approaches to game agent design, illuminating how various architectures address core challenges in the field.

Profiling Module Implementation

While agent profiling is widely recognized as a crucial component in agent frameworks, its practical implementation and empirical impact remain underexplored in contemporary systems.

CAB Profiling System Among the examined systems, CAB stands out by implementing explicit profiling through a character-based approach, defining the agent as "a soldier who must follow all the instructions the commander tells you" (Sanchez Llado, 2024). This implementation encompasses three key aspects: **demographic information** (soldier status), **personality traits** (disciplined, obedient), and **social context** (hierarchical relationship). However, CAB’s implementation lacks comparative analysis against an unprofiled baseline system, leaving the practical benefits of profiling largely unverified. This gap presents a significant research opportunity in agent architecture development.

WESE and NetPlay Profiling System In contrast, WESE and NetPlay Jeurissen et al. (2024, p. 3) deliberately omit character-specific profiling, instead emphasizing functional capabilities and role differentiation.

Memory Module Implementation

Netplay Memory System NetPlay implements a **unified memory** approach through a message timeline system, demonstrating straightforward application of in-context learning for short-term memory retention. The architecture categorizes information into system, AI, or human types, maintaining operational efficiency through a 500-token memory cap with first-in-last-out deletion (Jeurissen et al., 2024, Chapter III.A). While this *natural language storage* format preserves semantic richness, the limited context window constrains comprehensive memory integration.

CAB Memory System CAB’s architecture similarly employs a JSON-based messaging system for information management, representing another form of *natural language storage* without dedicated memory database capabilities. Both CAB (Sanchez Llado, 2024, p.38) and NetPlay (Jeurissen et al., 2024, Chapter III.A) integrate current game state into message prompts as context, potentially obviating the need for dedicated long-term memory as game state management is handled by the game system itself.

WESE Memory System WESE implements a more sophisticated **hybrid memory** structure through its knowledge graph system, explicitly modeling both short-term and long-term memory components. The system’s exploration agent actively extracts and generates knowledge triplets from environmental feedback, organizing them into a structured graph format. This implementation combines *database storage* advantages with efficient information retrieval mechanisms. The knowledge graph serves dual purposes: buffering current perceptions in short-term memory while consolidating critical information for long-term storage, enabling effective information utilization during extended task execution (Huang et al., 2024a, Chapter 3.2).

Planning Module

The examined systems demonstrate diverse approaches to planning, each implementing feedback-based mechanisms with varying degrees of sophistication.

NetPlay Planning System NetPlay implements a streamlined approach to **feedback-based planning**, utilizing **single-path reasoning** in a **zero-shot** implementation. The planning process integrates *chain-of-thought prompting* to process multiple input streams: task descriptions, environmental feedback, available skills, and output formats (Jeurissen et al., 2024, Chapter III.A). This comprehensive input processing enables effective decision-making without requiring pre-training, as illustrated in Figure 4.1.

WESE Planning System WESE demonstrates an advanced implementation of **feedback-based planning** through strategic separation of exploration and exploitation phases. The system employs *single-path reasoning* through Few-Shot ReAct prompting, directly integrating environmental feedback into its planning process (see prompting techniques, ??). This approach enables sophisticated knowledge accumulation and utilization during task execution (Huang et al., 2024a, Chapter 4.1).

CAB Planning System The CAB project implements **feedback-based planning** through a structured prompt-based system that showcases effective environmental feedback integration. The architecture’s planning process incorporates multiple components: character information, messages, tools, *Chain of Thought (CoT)*, and *Self-Ask* mechanisms. This comprehensive approach facilitates sophisticated task management and decision-making capabilities, effectively utilizing environmental feedback to guide planning decisions (Sanchez Llado, 2024, Chapter 3.2.2.1).

Action Module

The implementation of action modules varies significantly across the three systems, each addressing the challenges of environmental interaction through distinct approaches.

NetPlay Action System NetPlay implements its action capabilities through a skills-based tool system that bridges decision-making processes and environmental interaction. The system’s **action objectives** prioritize task completion through specific achievement markers, with environmental exploration facilitated through an "explore level" skill (Jeurissen et al., 2024, Table I). The system’s **action generation** operates through two primary mechanisms:

- *Plan-based generation*: Utilizes Chain-of-Thought prompting for skill selection, enabling high-level planning through predefined skills (Jeurissen et al., 2024, Chapter III.A(c))
- *Memory-based generation*: Models interaction history as a conversational stream, incorporating actions, thoughts, user feedback, and environmental responses

The **action space** comprises external tools implemented as NetHack command wrappers, organized into four categories: Special, Basic, Position, Inventory, and Direction commands. These enable environmental interaction through visible object manipulation, with spatial information handled by low-level skills.

WESE Action System WESE implements its action system through a knowledge graph structure, emphasizing memory-based action generation and information utilization. Its **action objectives** focus on information gathering and knowledge application in open-world environments, creating a knowledge-driven approach to problem-solving.

The system’s **action generation** leverages its knowledge graph-based memory system for information storage and retrieval. Action planning integrates extracted information into LLM prompts, while a "strong" agent utilizes stored knowledge for plan generation and task resolution.

The **action space** employs a tool-driven approach, with implementation varying by evaluation framework. For decision-making tasks in ALFworld and ScienceWorld, the action space consists of framework-specific predefined tools. For question answering against HotPotQA and FEVER datasets, the system implements three core commands (Huang et al., 2024a, Chapter 4.2):

CAB Action System CAB employs a structured action space through a JSON-formatted messaging system, enabling comprehensive task execution in labyrinth-based environments. Its **action objectives** emphasize task completion while maintaining effective user communication through a message-based command system.

The system’s **action generation** occurs through plan creation and atomic action decomposition, utilizing reasoning techniques such as Chain of Thought and Self-Ask. State information storage employs RAG-based retrieval for selective information access, enabling informed decision-making based on historical data.

Within its **action space**, CAB implements external tools for navigation and object manipulation, such as "Routing" and "PushObject" (Sanchez Llado, 2024, p. 37). These tools establish connections between LLM-generated responses and high-level actions, while internal reasoning capabilities through CoT and Self-Ask guide action selection and execution.

Table 4.1: Comparison of Agent Implementations Against Framework Requirements

Module	WESE	CAB	NetPlay
Profiling	Limited implementation; focuses on functional capabilities	Explicit character-based approach with defined role, traits, and hierarchy (Sanchez Llado, 2024)	Omits character-specific profiling; emphasizes functional roles (Jeurissen et al., 2024)
Memory	Hybrid memory with knowledge graph; combines short-term and long-term storage (Huang et al., 2024a)	JSON-based messaging system; natural language storage without dedicated database	Unified memory with 500-token timeline; FIFO deletion strategy
Planning	Feedback-based with exploration-exploitation separation; single-path reasoning	Feedback-based with CoT and Self-Ask mechanisms; comprehensive task management	Zero-shot implementation with single-path reasoning; chain-of-thought integration
Action	Knowledge graph-driven; tool-based implementation; framework-specific action space	JSON-formatted messaging; plan-based generation with RAG retrieval	Skills-based tool system; plan and memory-based generation; categorized command structure

Framework Implementation Summary Analysis of the three systems against the fundamental framework proposed by Wang et al. (2024) reveals varying degrees of alignment with theoretical requirements. WESE demonstrates the most sophisticated memory architecture through its hybrid implementation, aligning closely with the framework’s emphasis on differentiated memory structures. However, it shows limited implementation of profiling components, diverging from the framework’s recommendations for comprehensive agent identity definition.

CAB presents the most complete profiling implementation, incorporating all three key components identified in the framework: demographic information, personality traits, and social context (Sanchez Llado, 2024). Its planning module also closely follows framework guidelines by integrating multiple feedback mechanisms, though its memory implementation relies more heavily on immediate context than recommended.

NetPlay’s implementation emphasizes practical functionality over theoretical completeness, particularly evident in its streamlined memory system and focused action space. While this approach diverges from the framework’s comprehensive recommendations, it demonstrates effective task completion through specialized tool integration and clear action objectives (Jeurissen et al., 2024).

The comparative analysis suggests that while each system implements the core modules defined by Wang et al. (2024), they prioritize different aspects based on their specific use cases. This selective implementation approach indicates that practical constraints and application requirements often necessitate trade-offs between theoretical completeness and operational efficiency. Future development efforts might benefit from more balanced implementation of all framework components while maintaining the specialized advantages demonstrated by each system.

4.2.2 Evaluation Analysis

The evaluation approaches across the three systems reflect their distinct objectives and operational contexts. While this diversity presents challenges for direct comparison, it provides comprehensive insights into effective assessment methods for LLM-based game agents. This section analyzes each system’s evaluation methodology and key findings, highlighting both shared patterns and unique discoveries.

WESE Evaluation

WESE’s evaluation framework examines three critical performance dimensions: effectiveness (success rate), efficiency (step count), and computational cost (Huang et al., 2024a). The assessment spans two primary task categories: decision-making and question-answering scenarios.

Decision-Making Tasks In decision-making evaluation, WESE utilized two text-based environments: ALFWorld (Shridhar et al., 2021) and ScienceWorld (Wang et al., 2022). The evaluation compared WESE against Act and ReAct baselines, employing Llama-2-7B for weak exploration and text-davinci-003 for strong exploitation. To validate the decoupling strategy, researchers also tested a Strong Exploration to Strong Exploitation (SESE) variant.

Question-Answering Tasks For question-answering assessment, the evaluation leveraged HotPotQA (Yang et al., 2018) and FEVER (Thorne et al., 2018) datasets. The evaluation sampled 500 tasks from each development set and compared WESE against Chain of Thought (CoT), Act, and ReAct implementations, maintaining consistent metrics across all evaluations.

Results and Key Findings WESE demonstrated significant relative improvements, achieving a 26.94% increase in success rates for decision-making tasks while reducing computational costs by 53.83%. Question-answering tasks showed a 19.5% improvement compared to traditional approaches (Huang et al., 2024a).

However, absolute performance metrics reveal persistent challenges. In ALFWorld, even the most advanced implementation (ReAct-SESE) achieved a success rate of only 0.75, while the baseline Act method reached 0.43. The performance gap widened in ScienceWorld, where average rewards ranged from 15 to 18 out of 100 across all methods, indicating substantial room for improvement in complex environments.

The question-answering tasks demonstrated similar patterns. In HotPotQA, performance peaked with ReAct-SESE at 0.70, improving moderately from Act’s baseline of 0.56. FEVER proved more challenging, with success rates spanning from 0.296 to 0.416, highlighting the increased difficulty of fact verification tasks.

The evaluation revealed a clear trade-off between performance and computational efficiency. Enhanced architectures required substantial computational resources, processing over 7 million prompt tokens compared to baseline approaches using 2-3 million. While Chain of Thought approaches minimized step counts, they consistently demonstrated lower success rates, indicating a fundamental tension between efficiency and effectiveness.

These results indicate that while architectural improvements yielded relative gains, absolute performance remains moderate at best, with substantial computational overhead required for marginal improvements. This suggests that achieving high performance in complex environments continues to present significant challenges for LLM-based systems, despite advances in methodology.

CAB Evaluation

CAB’s evaluation framework encompasses three distinct testing environments (Sanchez Llado, 2024): the Obstacle Tower environment, a controlled recreation environment with procedurally generated mazes, and supplementary capability analyses. This comprehensive approach enabled assessment of both gaming performance and the impact of various LLM configurations.

Obstacle Tower Performance The evaluation in the Obstacle Tower environment followed competition protocols with five random runs, measuring success through floors completed before time or health expiration. The GPT-4-based agent achieved an average of 6.4 floors (SD = 1.9), surpassing the Random RL Agent (4.2 floors, SD = 1.3) but falling below human performance (9.3 floors, SD = 3.1) and competition leaders (10-19.4 floors). Traditional deep reinforcement learning approaches, including Proximal Policy Optimization (PPO) as an on-policy algorithm and Rainbow (RNB) as an off-policy algorithm, reached only 0.6-0.8 floors in both fixed and varied training environments. The agent exhibited strategic gameplay through resource collection but encountered limitations in RL navigation efficiency and puzzle-solving at floor ten.

Recreation Environment Testing A controlled testing environment using procedurally generated mazes evaluated fundamental capabilities across different model configurations. The testing environment varied maze complexity from 3×3 to 4×4 layouts and compared performance across multiple models including GPT-4, GPT-3.5, Llama3-8B, and Mixtral-8x7B. Feature ablation studies examined the impact of Chain of Thought reasoning, error recovery, and zero-shot capabilities, while also testing different prompt structures. Testing revealed GPT-4’s superior performance across configurations, while smaller models showed improved performance with disabled Chain of Thought reasoning (Sanchez Llado, 2024, p. 48).

Results and Key Findings Key findings revealed LLM-based agents’ advantages over traditional RL approaches, though they remained below specialized competition winners. Navigation efficiency and puzzle-solving emerged as critical performance bottlenecks, particularly in higher-level challenges. Model size correlated strongly with performance, though optimized configurations enabled effective performance from smaller models.

NetPlay

NetPlay’s evaluation focused on assessing the system’s NetHack gameplay capabilities and LLM-influenced performance characteristics (Jeurissen et al., 2024). The evaluation employed GPT-4-Turbo with specific parameters (temperature 0, JSON format, 500-token memory limit).

Full Game Performance The primary evaluation compared four agent variants using the Valkyrie role: Unguided agent (20 runs): Basic NetPlay implementation tasked to win without constraints Guided agent (10 runs): Enhanced with strategic principles from autoascend, including level-based progression and selective resource consumption Autoascend (100 runs): The winning agent from the 2021 NeurIPS Challenge, using a sophisticated behavior tree and internal state tracking, with specific strategies like staying on the first level until reaching experience level 8 Handcrafted agent (100 runs): A rule-based implementation following a fixed decision hierarchy (combat, healing, eating, item collection, exploration) to isolate LLM impact

Performance metrics revealed significant variations, with autoascend achieving superior results across all metrics (average score: 11341.94 ± 11625.39 , survival time: 21169.81 ± 9155.59 timesteps). The guided NetPlay version demonstrated notable improvements over its unguided counterpart, achieving higher average scores (405.00 ± 216.38 vs 284.85 ± 222.10) and experience levels (3.30 ± 0.95 vs 2.40 ± 1.23).

Scenario-Based Testing To evaluate the hypothesis that NetPlay fixates on straightforward approaches despite its creative potential, various small-scale scenarios tested game mechanics interaction, instruction following, and creative problem-solving. Five runs per scenario across all roles revealed that NetPlay excelled with concrete instructions, particularly in focused tasks like boulder manipulation and escape scenarios. However, performance degraded with ambiguous instructions, and the agent struggled with menu navigation despite understanding the correct actions, often failing due to missed procedural steps. These findings highlighted the system’s dependence on explicit feedback and detailed instruction, while demonstrating creative problem-solving capabilities when properly focused.

The scenario-based evaluation revealed clear patterns in the agent’s capabilities. In game mechanics scenarios, performance improved dramatically with explicit guidance, as demonstrated in the BAG and WAND tasks where success rates increased from 1/5 to 3/5 and 1/5 to 5/5 respectively. Instruction following showed strong performance in structured tasks, with perfect execution in ordered sequences and 80% success in conditional tasks. Creative problem-solving scenarios demonstrated the most striking improvements with guidance, particularly in the BOULDER task where success rates improved from 1/5 to 5/5 with explicit instructions. The ESCAPE scenarios showed moderate success (3/5) improving to perfect performance (5/5) when provided with environmental hints.

Key Findings The analysis reveals that while NetPlay falls significantly short of autoascend’s specialized performance, it demonstrates clear advantages over simple rule-based systems when properly guided. The high standard deviations in autoascend’s metrics suggest greater variability in its performance, while NetPlay shows more consistent, though lower, performance levels. The dramatic improvements seen in guided scenarios, particularly in complex tasks like ESCAPE and BOULDER, indicate that the system’s potential is heavily dependent on instruction clarity and task structure. These findings suggest

that future improvements should focus on bridging the gap between guided and unguided performance while maintaining the system’s ability to handle complex, multi-step tasks.

4.2.3 Key Findings and Limitations

The evaluation results across WESE, CAB, and NetPlay reveal several consistent patterns and shared challenges in LLM-based game agents, while also highlighting system-specific achievements and limitations.

Performance Bottlenecks The primary performance bottlenecks identified across systems include computational overhead and resource utilization. WESE’s enhanced architectures required substantial computational resources, processing over 7 million prompt tokens compared to baseline approaches using 2-3 million (Huang et al., 2024a). In CAB, navigation efficiency and puzzle-solving capabilities emerged as critical bottlenecks, particularly in higher-level challenges (Sanchez Llado, 2024). NetPlay demonstrated significant performance degradation with ambiguous instructions and struggled with menu navigation despite understanding the correct actions (Jeurissen et al., 2024).

Successful Strategies Several strategies proved effective across the systems. WESE’s decoupling approach achieved a 26.94% increase in success rates for decision-making tasks while reducing computational costs by 53.83% (Huang et al., 2024a). CAB showed that model size strongly correlated with performance, though optimized configurations enabled effective performance from smaller models (Sanchez Llado, 2024). NetPlay demonstrated that explicit guidance significantly improved performance, particularly in complex tasks, with success rates improving from 1/5 to 5/5 in specific scenarios when provided with clear instructions (Jeurissen et al., 2024).

Common Challenges Across all three systems, several common challenges emerged:

Absolute performance limitations: While relative improvements were achieved, absolute performance remained moderate. WESE’s most advanced implementation achieved success rates of only 0.75 in ALFWorld (Huang et al., 2024a), CAB remained below human performance levels (Sanchez Llado, 2024), and NetPlay fell significantly short of specialized systems like autoascend (Jeurissen et al., 2024).

Trade-offs between efficiency and effectiveness: WESE’s evaluation revealed a fundamental tension between computational efficiency and performance effectiveness, with Chain of Thought approaches minimizing step counts but showing lower success rates (Huang et al., 2024a).

Complexity handling: All systems showed degraded performance in more complex environments, with WESE’s performance in ScienceWorld achieving only 15-18 out of 100 points

(Huang et al., 2024a), CAB struggling with higher-level challenges (Sanchez Llado, 2024), and NetPlay requiring explicit guidance for complex tasks (Jeurissen et al., 2024).

Areas for Improvement The evaluations suggest several key areas for future development:

Computational Efficiency: There is a clear need to optimize resource utilization while maintaining performance levels, particularly in systems like WESE where computational overhead remains substantial (Huang et al., 2024a). **Autonomous Capability:** Improving systems’ ability to handle ambiguous or complex situations without explicit guidance, as highlighted by NetPlay’s performance variations between guided and unguided scenarios (Jeurissen et al., 2024). **Scalability:** Enhancing performance in complex environments while maintaining efficiency, as demonstrated by CAB’s challenges with higher-level puzzles and navigation (Sanchez Llado, 2024). **Bridging Performance Gaps:** Closing the gap between LLM-based systems and specialized solutions, particularly evident in NetPlay’s comparison with autoascend and CAB’s performance relative to competition winners (Jeurissen et al., 2024; Sanchez Llado, 2024).

Chapter 5: Implementation

This chapter presents the development and implementation of an autonomous agent system powered by large language models (LLMs), designed to interpret and execute natural language queries within Unity game environments. Building upon the theoretical foundations discussed in Section 2.1, our implementation demonstrates a practical realization of these concepts while addressing the specific challenges of game environment interaction.

5.1 System Architecture

Our system architecture implements the four essential modules identified in Section 2.2.2 - profiling, memory, planning, and action - while adapting them to the specific requirements of game environment interaction. The architecture consists of four primary components.

5.1.1 Core Components

Agent System

The agent system coordinates the interaction flow from natural language input to function execution. Following the ReAct paradigm discussed in Section ??, our implementation utilizes LangGraph for state management, enabling the agent to maintain context while processing requests and executing actions. This approach aligns with the reasoning and planning frameworks outlined in Section 2.4.

Knowledge Graph

The knowledge graph serves as the system's memory module, implementing both short-term and long-term memory capabilities as discussed in Section 2.3.2. Using Neo4j as its foundation, it captures the relationships between game objects, their components, and available methods. This implementation follows the property graph model described in Section 2.3.3, allowing nodes and edges to carry attribute-value pairs that represent the complex relationships within the game environment.

Tool System

The tool system realizes the action module capabilities described in Section 2.2.2, providing the agent with mechanisms for exploring and interacting with the game environment. Through specialized tools, it enables the agent to query the knowledge graph and execute methods using the method invocation system.

Method Invocation System

The method invocation system enables runtime execution of Unity methods through reflection-based invocation. This component implements comprehensive error handling to ensure safe method execution, addressing the validation challenges discussed in Section 2.3.5 regarding knowledge graph integration with executable systems.

Figure 5.1 illustrates the high-level architecture and component interactions.

Figure 5.1: System Architecture Overview

5.1.2 Technical Stack

Our implementation combines several key technologies to create a robust foundation for autonomous agent operation:

Language Model Framework

The system is built upon LangChain and LangGraph. This foundation enables the creation of agents that can maintain context, execute complex sequences of operations, and adapt to changing requirements during task execution.

Core Technologies

Unity and C# serve as the primary development platform, chosen for their comprehensive game development capabilities and strong integration features. The implementation leverages C#'s reflection system to enable dynamic method discovery and invocation while maintaining type safety.

Database Technology

Neo4j serves as the system’s graph database, selected based on its robust support for graph operations described in Section 2.3.2. Through Neo4j’s vector indexing features, we implement the semantic search capabilities discussed in Section 2.3.5, enabling efficient graph traversal and relationship queries.

5.2 Knowledge Graph

The knowledge graph serves as the semantic foundation of our system, bridging the gap between natural language understanding and Unity’s technical implementation. This section examines how we represent and maintain a comprehensive model of the game environment that supports both precise method lookup and semantic similarity search.

5.2.1 Design Considerations

Our knowledge graph implementation addresses several fundamental challenges in game environment representation. The primary challenge lies in creating a semantic bridge between Unity’s visual game elements and their underlying technical implementation. Game developers and players interact with Unity primarily through visual components and intuitive behaviors, while the underlying system operates on strongly-typed objects and methods.

5.2.2 Schema Design

The knowledge graph schema reflects both the structure of Unity’s component system and the semantic relationships needed for natural language processing. Our schema defines several core node types:

ObjectInstance Nodes

These nodes represent concrete instances of UnityEngine objects, which include `GameObject`, `Components` and other classes. Each `ObjectInstance` node maintains essential metadata:

- **name** (String): The name of the instance.
- **instanceId** (Integer): Unique identifier for the instance.
- **type** (String): The specific UnityEngine class type.
- **embedding** (Vector): Semantic embedding for similarity search.

Initially we used separate Node Types for GameObjects and Components, but found that having a general ObjectInstance node with a type property allows us to handle object references in a unified manner that also made the code easier to write.

Class Nodes

Defines a class, which is the script of the ObjectInstance:

- **name** (String): Class name.
- **isAbstract** (Boolean): Indicates if the class is abstract.
- **isSealed** (Boolean): Indicates if the class is sealed.
- **isStatic** (Boolean): Indicates if the class is static.

The class node primarily exist to map methods declaration, since those are not bound to the reference of an object. This way, we save on relationships between instance objects and methods, which would make the graph unübersichtlich.

Method Nodes

Method nodes capture the available behaviors in the system, including both technical details for execution and semantic information for natural language matching:

- **name** (String): Method name.
- **methodDescription** (String): Human-readable description.
- **token** (String): Unique identifier.
- **parameters** (String): Encodes parameter types and names.
- **returnType** (String): Specifies the return type.
- **accessLevel** (String): Access modifier (e.g., public, private).
- **embedding** (Vector): Semantic embedding.

Field Nodes

Field nodes represent the state variables of game objects, enabling state tracking and relationship modeling:

- **name** (String): Field name.
- **dataType** (String): Data type of the field.
- **value** (String): The current field value.
- **accessLevel** (String): Access modifier.
- **instanceId** (Integer): Unique identifier that references the ObjectInstance.

5.2.3 Relationships

The knowledge graph defines key relationships to model interactions and dependencies between entities:

ObjectInstance Relationships

- **HAS_COMPONENT** $\rightarrow$ *ObjectInstance*
Indicates that a ObjectInstance of Type "GameObject" possesses one or more ObjectInstance of Type "Component" instances defining its functionality.
Cardinality: 1:n (A GameObject may contain multiple components)
- **PARENT_OF** $\rightarrow$ *ObjectInstance*
Represents the hierarchical relationship between ObjectInstance of Type "GameObject", where one GameObject acts as a parent containing other subordinate GameObjects.
Cardinality: 1:n (A GameObject may have multiple child GameObjects)
- **INSTANCE_OF** $\rightarrow$ *Class*
Indicates that an ObjectInstance is an instantiation of a specific Class that defines its behavior and structure.
Cardinality: n:1 (Multiple ObjectInstances can be instances of the same Class)

Class Relationships

- **DECLARES_METHOD** $\rightarrow$ *Method*
Indicates that a Class declares one or more Method implementations defining its specific behavior.
Cardinality: 1:n (A Class may declare multiple methods)
- **DERIVES_FROM** $\rightarrow$ *Class*
Represents the inheritance hierarchy between classes by indicating that one Class derives from another.
Cardinality: n:1 (Multiple classes may derive from a single base class)

Field Relationships

- **REFERENCES** $\rightarrow$ *ObjectInstance*
Indicates that a Field within a ObjectReference maintains a reference to another ObjectReference, enabling interaction between classes and components.
Cardinality: n:1 (Multiple fields may reference the same Component)

5.2.4 Validation

The usage of reflection for dynamic method invocation brings dynamic capabilities to an otherwise static language. This, in theory, allows, for example, calling a method from another instance if the first instance has a reference to it. However, it also allows calling any method from within any context, which can lead to unsafe method invocations. To address this, we attempted to model C principles into the knowledge graph and query methods. However, modeling these principles proved more challenging than anticipated.

Neo4j, with its schemaless design, does not inherently enforce object-oriented constraints found in C#. This poses challenges in ensuring the correctness of method invocation and class relationships within our knowledge graph. While we have actively considered these issues and implemented validation mechanisms, we acknowledge that Neo4j is not the ideal solution for enforcing strict object-oriented rules.

Object-Oriented Principles in Graph Representation

Core object-oriented programming principles such as inheritance, polymorphism, and access control significantly influence method behavior and invocation patterns in C#. Ensuring that these principles are correctly represented in our knowledge graph requires a combination of structural modeling and query-based validation.

Inheritance Representation In our knowledge graph, class inheritance is represented using the `EXTENDS_CLASS` relationship:

```
1 (:Class_Derived) -[:EXTENDS_CLASS] -> (:Class_Base)
```

Listing 5.1: Graph Representation of Class Inheritance

Rather than redundantly linking each inherited method to derived classes, our query system dynamically determines inherited methods based on class hierarchy traversal. However, special attention is needed for private and protected methods, which should not be accessible outside their intended scope.

Access Control Implementation Access control modifiers define method visibility in C#. These modifiers are stored as properties on method nodes, allowing query-based enforcement of visibility rules:

```
1 (:Method {  
2   name: "CalculateDamage",  
3   accessModifier: "public"  
4 })
```

Listing 5.2: Graph Node with Access Modifier

Query templates ensure that methods retrieved during method invocation searches comply with their respective access control rules.

Polymorphism Representation Polymorphism, including method overriding and hiding, is implicitly represented in the graph through method signatures. Queries dynamically resolve overridden methods at retrieval time, ensuring that correct method versions are invoked when necessary.

Method Overloading Method overloading is modeled by treating each overloaded method as a distinct node with unique parameters and return types. This allows for accurate retrieval of methods based on argument matching.

Validation Challenges and Solutions

To mitigate the limitations of Neo4j’s schemaless nature, we have established a set of query templates that address validation concerns at the retrieval level. These templates, discussed in the Retrieval chapter, allow us to filter and validate method invocations before presenting them to the LLM. This approach ensures that only contextually appropriate information is surfaced, reducing the risk of incorrect method calls.

Ultimately, for simplicity’s sake, we decided to simplify the query process and allow otherwise unsafe invocations while creating validations and safeguards in the method invocation logic itself. This ensures that even if an unsafe method is retrieved, the execution environment enforces proper validation, preventing unintended behavior.

However, validation at the retrieval stage does not eliminate all potential issues. In cases where incorrect method invocations are made, the Unity system itself enforces runtime validation, preventing invalid calls from executing. While this safeguard ensures correctness at runtime, it also leads to an increased number of failed method invocations. Despite these trade-offs, this layered validation approach helps maintain a balance between flexibility and accuracy in method retrieval and execution.

5.2.5 Data Collection

The system implements a sophisticated data collection mechanism using reflection to capture both static structure and runtime state dynamically. This process ensures that the knowledge graph remains an accurate representation of the evolving Unity game environment.

The data collection process begins at the root `GameObject` and traverses its hierarchy, iterating through all child `GameObjects`. Each `GameObject`'s components are then examined, extracting information about their class implementation, fields, and methods. This hierarchical traversal guarantees a comprehensive snapshot of the scene structure.

To maintain synchronization with the evolving game state, data extraction is performed asynchronously within an update loop. This design allows continuous updates to reflect changes within the game environment in real time. However, this approach comes with performance trade-offs. The asynchronous nature introduces a slight latency in reflecting state changes in the knowledge graph. While the impact of this latency has not been measured quantitatively, it remains acceptable for the proof-of-concept stage and does not affect the core evaluation of this system.

The following code snippet demonstrates the data collection process for `GameObjects`:

```
1 private async Task<List> ExportGameObjectAsync(  
2   GameObject gameObject)  
3 {  
4   var queries = new List();  
5   int instanceId = gameObject.GetInstanceID();  
6  
7   var embedding = await GetEmbeddingAsync(gameObject.name);  
8  
9   queries.Add($@"  
10     MERGE (o:ObjectInstance {  
11       instanceId: {instanceId},  
12       type: '{gameObject.GetType().Name}',  
13       name: '{gameObject.name}'  
14     })");  
15  
16   // Component and field collection  
17   foreach (var component in gameObject.GetComponents<Component>())  
18   {  
19     var componentQueries =  
20       await ExportObjectInstanceAsync(component);  
21     queries.AddRange(componentQueries);  
22   }  
23  
24   return queries;  
25  
26 }
```

Listing 5.3: `GameObject` Data Collection Implementation

This method ensures that each `GameObject` and its components are accurately recorded in the knowledge graph, maintaining an up-to-date representation of the Unity scene structure.

5.2.6 Semantic Enhancement

Our implementation utilizes embedding models to prepare entities for efficient search operations. Neo4j requires the establishment of specialized indexes for rapid embedding processing and searching. These indexes are type-specific, constraining their application to particular node types. Given that method execution primarily requires identification of the executing actor and the method itself, we concentrated index creation on these two node types. Additional node types remain accessible through relationship queries.

The system generates embeddings through two primary approaches:

For methods, embeddings incorporate:

1. Method name
2. Method description

For Unity objects, embeddings consider:

1. Object names
2. Type information

```
1 string embeddingString = $"{methodInfo.Name} -  
   {graphMethodAttribute.Description}";  
2     var embedding = await  
       GetEmbeddingAsync(embeddingString);
```

Listing 5.4: Embedding Generation Implementation

5.3 Method Invocation System

The method invocation system enables runtime execution of Unity methods based on natural language commands. This section examines its design considerations and implementation details that ensure safe and reliable method execution.

5.3.1 Design Considerations

The method invocation system's design addresses three fundamental requirements for runtime method execution. First, it must safely execute methods on Unity objects while maintaining type safety and proper error handling. Second, it must handle parameter conversion between various data types, including both primitive types and Unity object references. Third, it must provide clear error feedback that can be meaningfully conveyed back to the language model for potential error recovery.

5.3.2 Object Reference and Data Type Resolution

At runtime, the system must handle two distinct types of data resolution: object references and primitive data types. For object references, the system uses Unity's `InstanceId` system, which provides a numerical identifier for each object in the scene. While a numerical identifier alone could be ambiguous, the combination with contextual information enables the language model to understand and manipulate object references effectively. The system resolves these references using Unity's `Resources.InstanceIDToObject` method, which returns the corresponding Unity object when provided with a valid `InstanceId`.

For primitive data types, the system implements type conversion to ensure parameters match their expected types. This process involves validating the input data against the method's parameter requirements and performing appropriate type conversions where necessary. The system handles both direct primitive type conversions (such as between numerical types) and more complex object reference resolutions.

5.3.3 Component Design

The method invocation system consists of three main components that work together to enable safe method execution:

The `MethodExecutor` class handles the complete lifecycle of method invocation, managing parameter validation, execution, and error handling. It provides the primary interface for method execution, receiving instructions and coordinating their processing.

The `MethodInstruction` class encapsulates all necessary information for method execution. As a data transfer object, it contains the target instance identifier, method name, and parameter array, providing a complete execution context while maintaining clear separation between invocation data and execution logic.

The `MethodExecutionException` class enables detailed error reporting while maintaining type safety. Through specialized error handling, it ensures that errors are properly categorized and can be effectively communicated back to the language model.

5.3.4 Error Handling

The system implements comprehensive error handling through several specific error types, each addressing a particular failure scenario:

- **OBJECT_NOT_FOUND** indicates that the referenced invocator object could not be found in the scene, typically due to an invalid `InstanceId`
- **METHOD_NOT_FOUND** signals that the specified method does not exist on the invocator object

- **PARAMETER_TYPE_MISMATCH** occurs when parameter conversion fails, indicating a type incompatibility
- **INVALID_OBJECT_PARAMETER** indicates that an object reference parameter could not be resolved to a valid Unity object
- **METHOD_EXECUTION_ERROR** represents general execution failures that occur during the method's actual execution

This detailed error categorization primarily serves the agent system's error recovery mechanisms, enabling it to make informed decisions about how to proceed when execution fails. The specific handling of these errors and recovery strategies will be discussed in detail in the Agent System section.

5.4 Retrieval System

The retrieval system serves as the intelligent interface between natural language queries and our knowledge graph, enabling efficient and semantically meaningful access to game environment information. This section examines how the system translates natural language intentions into precise database queries while maintaining both accuracy and performance.

5.4.1 Design Considerations

The development of the retrieval system addresses several fundamental challenges in bridging natural language understanding with structured data access. The system must handle the inherent ambiguity of natural language while maintaining the precision required for technical operations. Furthermore, it must balance query performance with semantic accuracy, ensuring that results are both relevant and timely.

Our implementation combines vector-based semantic search with traditional graph queries, enabling both fuzzy matching for natural language understanding and exact matching for technical operations. This hybrid approach allows the system to handle both direct commands and ambiguous requests effectively.

5.4.2 Vector Search Implementation

The system implements sophisticated vector-based search capabilities through Neo4j's vector index system. This implementation enables semantic similarity matching while maintaining efficient query performance.

Given a vector index, the system can perform a vector similarity search using cosine similarity. When a query is submitted, the embedding of the query is computed and compared against the stored embeddings of nodes in the graph database. The similarity

score determines the relevance of each node to the query, allowing the system to retrieve the most relevant game objects.

The following example demonstrates how a Python script can perform a vector similarity search to find relevant game objects based on an input query:

```
1 def find_gameobjects_by_vector_search(self, question, k=3):
2     """Search for game objects in the graph database and return metadata for the top k
3     results."""
4     _docs = self.objectinstance_index.similarity_search_with_relevance_scores(question,
5                                     k=k)
6     results = []
7
8     for document, score in _docs:
9         # Ensure the document is of type "GameObject"
10        if document.metadata.get("type") == "GameObject":
11            gameobject_metadata = document.metadata.copy() # Avoid modifying the original
12            metadata
13            gameobject_metadata["name"] = document.page_content # Add the name from
14            page_content
15            gameobject_metadata.pop("embedding", None) # Remove the embedding if it exists
16            gameobject_metadata["score"] = score # Include the relevance score
17            results.append(gameobject_metadata)
18
19 return results
```

Listing 5.5: Vector Search Implementation

The implementation maintains separate indexes for different entity types, allowing specialized search behavior for methods, game objects, and other elements. This separation enables context-appropriate similarity matching while maintaining query efficiency.

By leveraging Neo4j's vector index, the system efficiently finds related nodes based on their semantic meaning rather than just keyword matching. This is particularly useful for natural language queries, as it allows the system to retrieve relevant objects even if the query phrasing differs from the exact terms used in the database.

5.4.3 Query Generation

The system implements a sophisticated query generation system that translates natural language intentions into executable Cypher queries. This process involves several stages of transformation and validation:

```
1 def run_query_with_vector_context(self, cypher_llm, template, query):
2     method_results =
3         self.method_index.similarity_search_with_relevance_scores(query, k=3)
4     objectinstance_results =
5         self.objectinstance_index.similarity_search_with_relevance_scores(query,
6                                     k=3)
```

```

4     all_results = method_results + objectinstance_results
5
6     if not all_results:
7         return {"error": "No relevant nodes found in vector search."}
8
9     context_str = "\n".join([
10         f"Node: {document.page_content}, Properties: {document.metadata}, Score: {score}"
11         for document, score in all_results
12     ])
13
14     context_str = context_str.replace("{", "{{").replace("}", "}}")
15     partial_prompt = template.replace("{context}", context_str)
16
17     CYPHER_GENERATION_PROMPT = PromptTemplate(
18         input_variables=["schema", "query"], template=partial_prompt
19     )
20
21     qa_chain = GraphCypherQAChain.from_llm(
22         graph=self.neo4j_graph,
23         llm = cypher_llm,
24         # cypher_llm=cypher_llm,
25         # qa_llm=qa_llm,
26         verbose=True,
27         cypher_prompt=CYPHER_GENERATION_PROMPT,
28         validate_cypher=True,
29         return_direct=True,
30         allow_dangerous_requests=True,
31     )
32
33     answer = qa_chain.invoke({"query": query})
34
35     return answer
36 }

```

Listing 5.6: Query Generation Implementation

The query generation process incorporates both semantic context from vector search and structural information from the graph schema. This combination enables the system to generate queries that are both semantically relevant and technically valid.

5.4.4 Query Templates

Cypher query generation works well for simpler queries but struggles with more complex ones. As a result, for specific and complicated use cases, we utilize predefined query templates and post-filtering techniques to enforce certain rules. This approach helps maintain correctness and efficiency in querying the knowledge graph.

```

1 def get_all_paths_to_methods_from_gameobject(self, dynamic, game_object_id):
2     """Retrieve all methods callable from a given GameObject."""
3     callable_methods = []
4     with self.neo4j_db.session() as session:
5         if dynamic:
6             result = session.run(
7                 """
8                 MATCH (g:ObjectInstance {instanceId: $game_object_id})
9                 WHERE g.type = 'GameObject'
10                MATCH p = (g)-[r*]->(m:Method)
11                RETURN DISTINCT p
12                """,
13                game_object_id=game_object_id
14            )
15
16            for record in result:
17                path = record["p"]
18                nodes = path.nodes
19                # Use the revised validation function
20                if self.validate_callable_method_path(nodes):
21                    callable_methods.append(nodes[-1]) # Last node is the Method
22            else:
23                result = session.run(
24                    """
25                    MATCH (g:ObjectInstance {instanceId: $game_object_id})
26                    WHERE g.type = 'GameObject'
27                    MATCH p = (g)-[r*]->(m:Method)
28                    WHERE NONE(rel IN r WHERE type(rel) = 'REFERENCES')
29                    RETURN DISTINCT p
30                    """,
31                    game_object_id=game_object_id
32                )
33
34                for record in result:
35                    path = record["p"]
36                    nodes = path.nodes
37                    callable_methods.append(nodes[-1])
38
39 return callable_methods

```

Listing 5.7: Retrieving Methods Callable from a GameObject

5.5 Agent

The development of an agent system for game environments necessitates a specialized approach that accounts for the inherent dynamics and complexity of game worlds. Our system builds upon the ReAct (Reason-Act-Observe) architecture, enhanced through LangGraph

integration. This architectural decision enables the agent to not only execute actions but also reflect on their outcomes and adapt its behavior accordingly.

```

1 builder = StateGraph(AgentState)
2
3 builder.add_node("agent", call_model)
4 builder.add_node("tools", tool_node)
5
6 builder.add_edge(START, "agent")
7 builder.add_conditional_edges(
8     "agent",
9     should_continue,
10    ["tools", END])
11 builder.add_edge("tools", "agent")
12
13 graph = builder.compile()
```

Listing 5.8: System Integration

5.5.1 Design Considerations

One of the key design considerations is that method invocation is dynamic: the state of the game may allow or restrict the invocation of a method by a certain object at any given moment. Additionally, the number of available methods in the system can be vast. To optimize efficiency, the agent must selectively filter and prioritize relevant methods, streamlining decision-making and improving execution performance.

Our implementation must also account for asynchronous execution. This is particularly relevant when managing interactions between different tools and responding to evolving game states. By structuring our system to handle asynchronous method calls and responses effectively, we ensure robust and scalable performance.

Dynamic Action Space Management

Game environments present a unique challenge where available actions and methods can change dynamically, especially when new game mechanics are introduced or game states evolve. Our architecture implements a flexible tool management system that dynamically recognizes and integrates action possibilities, allowing for adaptive agent behavior.

5.5.2 State Management Architecture

LangGraph operates on state machines, which aligns naturally with our agent's architecture. While state machines are a well-established paradigm, our implementation refines this approach by introducing a three-tier state management hierarchy. This separation enhances modularity and ensures that different components interact seamlessly.

```
1 class AgentState(TypedDict):
2     messages: Annotated[list, add_messages]
3     static_tools: List
4     dynamic_tools: Dict
5     llm: None
6     invoke_durations: List[float]
7     remaining_steps: RemainingSteps
```

Listing 5.9: Agent State Definition

This state structure clearly delineates conversation context, tool availability, and execution tracking. By maintaining well-defined interfaces, our implementation facilitates efficient state transitions and persistent state management.

5.5.3 Tool Management

The system categorizes tools into two distinct sets: - **Static Tools (Explorative Tools):** Always available to the agent, these tools support game state exploration and facilitate communication with the user. - **Dynamic Tools (Function Tools):** Generated at runtime based on method signatures extracted from method nodes in Neo4j. These tools allow the agent to adapt to evolving game mechanics.

```
1 def tool_node(state: AgentState):
2     dynamic_tools = state.get('dynamic_tools', {})
3     static_tools = state.get('static_tools', [])
4     combined_tools = static_tools + list(dynamic_tools.values())
5     tools_by_name = {tool.name.strip(): tool
6                     for tool in combined_tools}
7
8     for tool_call in state["messages"][-1].tool_calls:
9         tool_name = tool_call["name"].strip()
10        try:
11            if tool_name not in tools_by_name:
12                raise KeyError(
13                    f"Tool '{tool_name}' not found in tools_by_name")
14
15            start_time = time.perf_counter()
16            tool_result = tools_by_name[tool_name].invoke(
17                tool_call["args"])
18            duration = time.perf_counter() - start_time
19
20            # Tool execution and result processing...
21        except Exception as e:
22            # Error handling...
```

Listing 5.10: Tool Management Implementation

Within the tool node, tool selection dictates whether the system generates a dynamic function tool or processes the output of an existing tool. This dynamic approach enables our agent to seamlessly integrate new functionality based on the evolving game state.

5.5.4 Error Handling

The system implements comprehensive error handling to maintain execution integrity: - Errors are categorized to ensure meaningful recovery strategies. - Context is preserved to facilitate immediate troubleshooting and long-term analysis. - The error pipeline integrates with the agent’s state management, enabling informed decision-making when failures occur.

By employing structured error handling, our agent system enhances robustness and ensures stable performance across varying game environments.

5.6 Tool Design

The tools leverage the LLM’s inherent capabilities for tool selection, enabling efficient exploration and utilization of game functions. By establishing a standardized interface, we present Unity functions as compliant tools, allowing the LLM to invoke functions on our behalf.

As mentioned earlier in the agent section, tools are categorized into static (explorative) and dynamic tools. This classification optimizes the tool selection process, ensuring that only relevant tools are available, thereby enhancing LLM decision-making.

5.6.1 Explorative Tools

We utilize Cypher query templates to identify related methods and game objects capable of invoking them at the time of retrieval. This process involves extracting actions and verbs from the input query and performing vector similarity searches. These specialized tools are exclusively designed for method discovery, ensuring the return of method tokens necessary for dynamic tool creation.

Additionally, we employ a Cypher conversion tool, which aids in answering general questions about the game state. It also serves as a fallback tool in cases where insufficient or incorrect details are provided for function tools.

5.6.2 Function Tools

Function tools are dynamically created to invoke methods on behalf of the LLM. For proper execution, they must conform to the OpenAI tool schema. The method for generating these tools is outlined below.

Dynamic Tool Creation

The following listing provides a structured overview of the tool creation process, suitable for inclusion in the appendix.

5.6.3 Function Calling

The function execution process follows the same structured approach outlined above. This ensures proper invocation and seamless communication with Unity by passing the necessary information.

Chapter 6: Evaluation

6.0.1 Implementation Details

The evaluation framework was implemented using Python, incorporating various libraries for data processing and analysis. The system interfaces with a Unity server through REST API calls and utilizes Neo4j for graph database operations.

6.1 Model Selection

6.1.1 Requirements and Infrastructure

The selection of language models for this evaluation was driven by both technical requirements and practical considerations. To ensure consistent implementation and evaluation across different model providers, we utilized OpenRouter as a unified interface for LLM access. This approach eliminated the need for multiple provider-specific implementations in Langchain, which would have increased complexity and potential inconsistencies in the evaluation process.

The technical requirements for model selection included:

- Support for the "tools" parameter enabling function calling capabilities
- Support for structured output generation
- Compatibility with the OpenAI API format through OpenRouter

These requirements notably excluded some promising models, such as DeepSeek R1 and its variants, which did not support the required functionality at the time of evaluation.

6.1.2 Selected Models

The selected models represent a diverse range of capabilities and cost structures. While not all model providers disclose their parameter counts—notably Google’s Gemini series and OpenAI’s GPT series—we can use the input token pricing as a proxy for model size and complexity, as models with more parameters typically command higher inference costs.

The evaluation included the following models, ordered by input token pricing (from highest to lowest):

Model	Input Cost (per million tokens)	Output Cost (per million tokens)
Cohere Command-R Plus	\$2.85	\$14.25
Mistral Large 24-11	\$2.00	\$6.00
OpenAI O3-Mini	\$1.10	\$4.40
Meta LLaMA-3.3 70B Instruct	\$0.12	\$0.30
Google Gemini 2.0 Flash	\$0.10	\$0.40

Table 6.1: Model pricing comparison

6.2 Evaluation Metrics

6.2.1 Metric Definitions and Calculations

Basic Success Metrics

- **Success Rate:** Ratio of fully successful test cases to total test cases

$$\text{Success Rate} = \frac{\text{number of successful tests}}{\text{total number of tests}} \quad (6.1)$$

- **Partial Success Rate:** Ratio of partially successful test cases to total test cases

$$\text{Partial Success Rate} = \frac{\text{number of partially successful tests}}{\text{total number of tests}} \quad (6.2)$$

Duration Metrics

For each model, we track:

- **Total Duration:** Sum of all time spent

$$\text{Total Duration} = \text{accumulated_tool_duration} + \text{accumulated_inference_duration} \quad (6.3)$$

- **Tool Duration Ratio:** Proportion of time spent in tool operations

$$\text{Tool Duration Ratio} = \frac{\text{accumulated_tool_duration}}{\text{total_duration}} \quad (6.4)$$

- **Inference Duration Ratio:** Proportion of time spent in model inference

$$\text{Inference Duration Ratio} = \frac{\text{accumulated_inference_duration}}{\text{total_duration}} \quad (6.5)$$

Error Metrics

- **Average Errors per Test:**

$$\text{Avg Errors} = \frac{\text{total number of errors}}{\text{total number of tests}} \quad (6.6)$$

- **Error Type Distribution:** Frequency count of each error type encountered

$$\text{Error Rate}_{\text{type}} = \frac{\text{count of specific error type}}{\text{total error count}} \quad (6.7)$$

Cost-Efficiency Metrics

- **Average Cost per Test:**

$$\text{Avg Cost} = \frac{\text{total_cost_in_dollars}}{\text{number of tests}} \quad (6.8)$$

- **Cost per Success:** Cost required per successful test case

$$\text{Cost per Success} = \frac{\text{total_cost_in_dollars}}{\text{number of successful tests}} \quad (6.9)$$

Note: If there are no successful tests, Cost per Success is undefined.

- **Success Rate per Dollar:** Number of successes achieved per dollar spent

$$\text{Success per \$} = \frac{\text{success_rate}}{\text{avg_cost_per_test}} \quad (6.10)$$

Note: This metric can be unintuitive due to very small dollar amounts. A more practical interpretation would be:

$$\text{Success per \$100} = \frac{\text{success_rate} \times 100}{\text{avg_cost_per_test} \times 100} \quad (6.11)$$

Token Efficiency Metrics

- **Token Statistics:**

$$\text{Token Stats} = \{\text{mean, median, std, min, max}\} \quad (6.12)$$

- **Token Efficiency Ratio:** Ratio of output tokens to input tokens

$$\text{Token Efficiency} = \frac{\text{output_tokens}}{\text{input_tokens}} \quad (6.13)$$

- **Processing Rate:** Tokens processed per second

$$\text{Tokens/Second} = \frac{\text{input_tokens} + \text{output_tokens}}{\text{total_duration}} \quad (6.14)$$

Error Analysis

- **Error distribution:** Types and frequencies of errors per model
- **Error rates:** Mean number of errors and total error count per model

6.3 Results

6.3.1 Quantitative

Table 6.2: Model Performance Metrics

Model	Success Rate	Partial Success	Mean Cost (\$)	Cost Efficiency
command-r-plus-08-2024	5.00%	5.00%	0.0123	407.83%
gemini-2.0-flash-001	65.00%	80.00%	0.0008	82929.85%
llama-3.3-70b-instruct	45.00%	95.00%	0.0012	37724.08%
mistral-large-2411	50.00%	80.00%	0.0174	2868.85%
o3-mini	50.00%	95.00%	0.0227	2205.96%

Table 6.3: Token Usage Statistics

Model	Mean	Median	Std Dev	Min	Max
<i>Input Tokens</i>					
command-r-plus-08-2024	4,108	3,940	3,764	1,084	18,704
gemini-2.0-flash-001	6,906	5,468	4,044	1,252	15,275
llama-3.3-70b-instruct	8,731	7,801	2,797	359	14,708
mistral-large-2411	7,478	6,493	3,109	5,085	14,759
o3-mini	5,142	4,940	1,248	3,769	9,177
<i>Output Tokens</i>					
command-r-plus-08-2024	264	226	163	92	773
gemini-2.0-flash-001	233	184	156	29	583
llama-3.3-70b-instruct	484	353	366	0	1,327
mistral-large-2411	412	324	230	270	1,265
o3-mini	3,866	3,620	1,460	1,726	7,575

Table 6.4: Duration Metrics (seconds)

Model	Avg Total	Avg Tool	Avg Inference	Total Duration
command-r-plus-08-2024	9.15	1.10	7.91	182.92
gemini-2.0-flash-001	9.32	2.26	6.88	186.43
llama-3.3-70b-instruct	28.29	16.55	11.56	565.89
mistral-large-2411	11.50	5.55	5.63	229.93
o3-mini	51.26	22.07	29.04	1025.24

Table 6.5: Error Distribution by Model

Error Type	command-r-plus-08-2024	gemini-2.0-flash-001	llama-3.3-70b-instruct	mistral-large-2411	o3-mini
UNKNOWN_ERROR	11	1	23	1	1
PARAMETER_TYPE_MISMATCH	4	9	4	3	0
INVOCATION_ERROR	1	0	0	0	0
GAMEOBJECT_NOT_FOUND	1	0	4	0	0
Total Errors	17	10	31	4	1
Avg. Errors/Test	8.50	5.00	15.50	2.00	0.50

6.4 Discussion

6.4.1 Interpretation of Results

The comparative analysis of five language models reveals several notable patterns in terms of cost efficiency, performance, and reliability. The most striking finding is the significant variance in success rates, with Gemini-2.0-flash-001 achieving the highest baseline success rate at 65%, while command-r-plus-08-2024 demonstrated notably lower performance at 5%. When considering partial successes, both Llama-3.3-70b-instruct and o3-mini showed impressive adaptability, reaching 95% partial success rates.

Cost efficiency analysis presents an interesting trade-off between performance and operational expenses. While Gemini-2.0-flash-001 offers the lowest average cost per interaction (\$0.000784), it demonstrates relatively high cost-per-success metrics when normalized to \$100 (8.29%). This suggests that raw operational costs may not be the most reliable indicator of model efficiency. The o3-mini model, despite having the highest average cost (\$0.022666), achieves one of the best cost-efficiency ratios when accounting for its success rate.

Performance metrics reveal distinct operational characteristics across models. Token usage patterns show that Llama-3.3-70b-instruct processes the highest mean input tokens (8,731), while o3-mini generates significantly more output tokens (mean: 3,866) compared to other models. Duration metrics highlight substantial differences in processing speed, with o3-mini

requiring the longest average total duration (51.26 seconds) while command-r-plus-08-2024 and Gemini-2.0-flash-001 complete tasks in approximately 9 seconds.

Error analysis provides crucial insights into model reliability. Mistral-large-2411 and o3-mini demonstrate superior error resistance with only 4 and 1 total errors respectively, while Llama-3.3-70b-instruct shows higher error susceptibility with 31 total errors. Notably, command-r-plus-08-2024's unusually low performance (5% success rate) can be attributed to a high number of "UNKNOWN_ERROR" occurrences, which subsequent analysis revealed to be primarily Neo4j Cypher invocation errors that weren't properly categorized. This model's specific difficulty with Cypher queries explains its unexpectedly poor performance in the test suite.

6.4.2 Implications

Practical Implications

These findings have several practical implications for model deployment and selection:

The observed latency metrics raise significant concerns for interactive applications, particularly in gaming environments. With average duration times ranging from 9.15 to 51.26 seconds per request, these response times exceed acceptable thresholds for player interaction. The high latency is attributed to both inference time and tool usage, which contribute equally to the overall delay in some cases. For gaming applications, where immediate feedback is crucial for player engagement, these duration metrics indicate a need for substantial optimization or alternative approaches.

Cost optimization strategies should consider success rates alongside raw operational costs. The apparent cost advantage of cheaper models may be offset by their lower success rates and higher error frequencies.

Error patterns indicate a need for robust error handling mechanisms, particularly for `PARAMETER_TYPE_MISMATCH` errors, which appear consistently across models.

6.4.3 Future Research Directions

Several promising areas for future research emerge from this analysis:

The current metrics demonstrate that while most models can achieve partial success in various areas, they consistently struggle to fulfill all subgoals within a single input query. This pattern suggests a need for research into improving models' ability to handle multi-objective tasks comprehensively, rather than addressing subtasks in isolation. Error analysis reveals two critical limitations in current model implementations: insufficient contextual information for accurate inference generation, and an inability to properly differentiate between instance IDs across different types, leading to parameter type mismatches. Future research

should focus on developing more sophisticated type-aware architectures and improving context handling mechanisms. Development of more sophisticated cost-efficiency metrics that better capture the relationship between model performance and operational expenses, particularly in the context of partial success scenarios. Investigation into reducing the latency impact of both inference and tool usage, possibly through the development of specialized gaming-oriented model architectures that prioritize response speed. Research into optimization techniques for reducing token usage while maintaining performance, particularly for models showing high input token consumption. Exploration of hybrid approaches that leverage the strengths of different models for specific task types, potentially allowing for dynamic model selection based on subtask requirements.

List of Figures

2.1	"A unified framework for the architecture design of LLM-based autonomous agent." Source: Wang et al., 2024	8
2.2	"A generic RAG architecture. The user queries, spanning different modalities, serve as input to both the retriever and the generator. The retriever extracts relevant information from data sources. The generator interacts with the retrieval results and ultimately produces outcomes of various modalities." Source: Zhao et al., 2024	12
2.3	"Directed-edge labelled graph describing events and their venues." Source: Hogan et al., 2021	14
2.4	"Data about capitals and countries in a del graph and a heterogeneous graph." Source: Hogan et al., 2021	15
2.5	"Flight data in a del graph and a property graph." Source: Hogan et al., 2021	15
4.1	"Illustration of NetPlay playing NetHack. The process involves constructing a prompt using messages representing past events, the current observation, and a task description containing available skills and the desired output format. The response is parsed to retrieve the next skill. While executing the selected skill, a tracker enriches the given observations and detects important events, such as when a new monster appears. When the skill is done, or events interrupt the skill execution, the agent will restart the prompting process." Source: Jeurissen et al., 2024, Figure 2	26
4.2	"Framework of WESE. The left part represents the weak exploration and the right part represents the strong exploitation. We employ Llama-2-7B as the weak agent and text-davinci-003 as the strong agent in the implementation." Source: Huang et al., 2024a, Figure 2	26
4.3	"General architecture of the LLM agent. The agent in the game environment requests the LLM for an action through a prompt. Then the agent attempts to parse the response, and after attempting the action, it starts the next requests." Source: Sanchez Llado, 2024, Figure 3.4	27
5.1	System Architecture Overview	40

List of Tables

4.1	Comparison of Agent Implementations Against Framework Requirements .	31
6.1	Model pricing comparison	58
6.2	Model Performance Metrics	60
6.3	Token Usage Statistics	60
6.4	Duration Metrics (seconds)	60
6.5	Error Distribution by Model	61

Listings

5.1	Graph Representation of Class Inheritance	44
5.2	Graph Node with Access Modifier	44
5.3	GameObject Data Collection Implementation	46
5.4	Embedding Generation Implementation	47
5.5	Vector Search Implementation	50
5.6	Query Generation Implementation	50
5.7	Retrieving Methods Callable from a GameObject	52
5.8	System Integration	53
5.9	Agent State Definition	54
5.10	Tool Management Implementation	54

Bibliography

- Jurafsky, D., & Martin, J. H. (2024). *Speech and language processing: An introduction to natural language processing, computational linguistics, and speech recognition with language models* (3rd) [Online manuscript released August 20, 2024]. <https://web.stanford.edu/~jurafsky/slp3/>
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2023). Attention is all you need. <https://arxiv.org/abs/1706.03762>
- Matarazzo, A., & Torlone, R. (2025). A survey on large language models with some insights on their capabilities and limitations. <https://arxiv.org/abs/2501.04040>
- Jiang, B., Xie, Y., Hao, Z., Wang, X., Mallick, T., Su, W. J., Taylor, C. J., & Roth, D. (2024). A peek into token bias: Large language models are not yet genuine reasoners. <https://arxiv.org/abs/2406.11050>
- Shi, F., Chen, X., Misra, K., Scales, N., Dohan, D., Chi, E. H., Schärli, N., & Zhou, D. Large language models can be easily distracted by irrelevant context (A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett, Eds.). In: *Proceedings of the 40th international conference on machine learning* (A. Krause, E. Brunskill, K. Cho, B. Engelhardt, S. Sabato, & J. Scarlett, Eds.). Ed. by Krause, A., Brunskill, E., Cho, K., Engelhardt, B., Sabato, S., & Scarlett, J. 202. Proceedings of Machine Learning Research. PMLR, 2023, 31210–31227. <https://proceedings.mlr.press/v202/shi23a.html>
- Mirzadeh, I., Alizadeh, K., Shahrokhi, H., Tuzel, O., Bengio, S., & Farajtabar, M. (2024). Gsm-symbolic: Understanding the limitations of mathematical reasoning in large language models. <https://arxiv.org/abs/2410.05229>
- Huang, X., Liu, W., Chen, X., Wang, X., Wang, H., Lian, D., Wang, Y., Tang, R., & Chen, E. (2024b). Understanding the planning of llm agents: A survey. <https://arxiv.org/abs/2402.02716>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6). <https://doi.org/10.1007/s11704-024-40231-1>
- Wooldridge, M., & Jennings, N. R. (1995). Intelligent agents: Theory and practice. *The Knowledge Engineering Review*, 10, 115 –152. <https://api.semanticscholar.org/CorpusID:221342993>
- Zhao, P., Zhang, H., Yu, Q., Wang, Z., Geng, Y., Fu, F., Yang, L., Zhang, W., Jiang, J., & Cui, B. (2024). Retrieval-augmented generation for ai-generated content: A survey. <https://arxiv.org/abs/2402.19473>

- Mallen, A., Asai, A., Zhong, V., Das, R., Khashabi, D., & Hajishirzi, H. (2023). When not to trust language models: Investigating effectiveness of parametric and non-parametric memories. <https://arxiv.org/abs/2212.10511>
- Izacard, G., Lewis, P., Lomeli, M., Hosseini, L., Petroni, F., Schick, T., Dwivedi-Yu, J., Joulin, A., Riedel, S., & Grave, E. (2022). Atlas: Few-shot learning with retrieval augmented language models. <https://arxiv.org/abs/2208.03299>
- Wu, Y., Rabe, M. N., Hutchins, D., & Szegedy, C. (2022). Memorizing transformers. <https://arxiv.org/abs/2203.08913>
- He, Z., Zhong, Z., Cai, T., Lee, J. D., & He, D. (2024). Rest: Retrieval-based speculative decoding. <https://arxiv.org/abs/2311.08252>
- Kang, M., Gürel, N. M., Yu, N., Song, D., & Li, B. (2024). C-rag: Certified generation risks for retrieval-augmented language models. <https://arxiv.org/abs/2402.03181>
- Hogan, A., Blomqvist, E., Cochez, M., D’amato, C., Melo, G. D., Gutierrez, C., Kirrane, S., Gayo, J. E. L., Navigli, R., Neumaier, S., Ngomo, A.-C. N., Polleres, A., Rashid, S. M., Rula, A., Schmelzeisen, L., Sequeda, J., Staab, S., & Zimmermann, A. (2021). Knowledge graphs. *ACM Computing Surveys*, 54(4), 1–37. <https://doi.org/10.1145/3447772>
- Serles, U., & Fensel, D. (2024). *An introduction to knowledge graphs*. Springer Nature Switzerland. <https://doi.org/10.1007/978-3-031-45256-7>
- Xu, Z., Cruz, M. J., Guevara, M., Wang, T., Deshpande, M., Wang, X., & Li, Z. Retrieval-augmented generation with knowledge graphs for customer service question answering. In: *Proceedings of the 47th international acm sigir conference on research and development in information retrieval*. SIGIR 2024. ACM, 2024, July, 2905–2909. <https://doi.org/10.1145/3626772.3661370>
- Xu, K., Feng, Y., Huang, S., & Zhao, D. Hybrid question answering over knowledge base and free text (Y. Matsumoto & R. Prasad, Eds.). In: *Proceedings of COLING 2016, the 26th international conference on computational linguistics: Technical papers* (Y. Matsumoto & R. Prasad, Eds.). Ed. by Matsumoto, Y., & Prasad, R. Osaka, Japan: The COLING 2016 Organizing Committee, 2016, December, 2397–2407. <https://aclanthology.org/C16-1226/>
- Bordes, A., Weston, J., & Usunier, N. (2014). Open question answering with weakly supervised embedding models. <https://arxiv.org/abs/1404.4326>
- Unger, C., Bühmann, L., Lehmann, J., Ngonga Ngomo, A.-C., Gerber, D., & Cimiano, P. (2012). Template-based question answering over rdf data. *WWW’12 - Proceedings of the 21st Annual Conference on World Wide Web*. <https://doi.org/10.1145/2187836.2187923>
- Emonet, V., Bolleman, J., Duvaud, S., de Farias, T. M., & Sima, A. C. (2024). Llm-based sparql query generation from natural language over federated knowledge graphs. <https://arxiv.org/abs/2410.06062>
- Kojima, T., Gu, S. S., Reid, M., Matsuo, Y., & Iwasawa, Y. (2023). Large language models are zero-shot reasoners. <https://arxiv.org/abs/2205.11916>
- Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G.,

- Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., . . . Amodei, D. (2020). Language models are few-shot learners. <https://arxiv.org/abs/2005.14165>
- Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., & Zhou, D. (2023). Chain-of-thought prompting elicits reasoning in large language models. <https://arxiv.org/abs/2201.11903>
- Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., & Cao, Y. (2023). React: Synergizing reasoning and acting in language models. <https://arxiv.org/abs/2210.03629>
- Jeurissen, D., Perez-Liebana, D., Gow, J., Cakmak, D., & Kwan, J. (2024). Playing nethack with llms: Potential limitations as zero-shot agents. <https://arxiv.org/abs/2403.00690>
- Huang, X., Liu, W., Chen, X., Wang, X., Lian, D., Wang, Y., Tang, R., & Chen, E. (2024a). Wese: Weak exploration to strong exploitation for llm agents. <https://arxiv.org/abs/2404.07456>
- Sanchez Llado, F. (2024). *Controlling agents behaviours through llms* (Publication No. 2024:450) [Master’s thesis, KTH, School of Electrical Engineering and Computer Science (EECS)]. KTH Royal Institute of Technology.
- Shridhar, M., Yuan, X., Côté, M.-A., Bisk, Y., Trischler, A., & Hausknecht, M. (2021). Alfworld: Aligning text and embodied environments for interactive learning. <https://arxiv.org/abs/2010.03768>
- Wang, R., Jansen, P., Côté, M.-A., & Ammanabrolu, P. (2022). Scienceworld: Is your agent smarter than a 5th grader? <https://arxiv.org/abs/2203.07540>
- Yang, Z., Qi, P., Zhang, S., Bengio, Y., Cohen, W. W., Salakhutdinov, R., & Manning, C. D. (2018). Hotpotqa: A dataset for diverse, explainable multi-hop question answering. <https://arxiv.org/abs/1809.09600>
- Thorne, J., Vlachos, A., Christodoulopoulos, C., & Mittal, A. (2018). Fever: A large-scale dataset for fact extraction and verification. <https://arxiv.org/abs/1803.05355>

Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Masterarbeit selbstständig und nur unter Verwendung der angegebenen Quellen und Hilfsmittel verfasst habe. Die Arbeit wurde bisher in gleicher oder ähnlicher Form keiner anderen Prüfungsbehörde vorgelegt.

Berlin, den 21.02.2025

Ruslan Novikov